```
import { createClientFromRequest } from 'npm:@base44/sdk@0.5.0';

// --- START: SHARED YAHOO SESSION MANAGEMENT ---
// Reuse the same session management from syncRecentPricesYahoo
let yahooSession = {
  cookie: null,
  crumb: null,
  acquiredAt: 0,
  inFlight: null
};

const SESSION_TTL_MS = 60 * 60 * 1000; // 60 minutes
let tokenBucket = {
  tokens: 2,
  lastRefill: Date.now(),
  refillRate: 1 / 600, // 1 token per 600ms
  maxTokens: 2
};

function refillTokens() {
  const now = Date.now();
  const elapsed = now - tokenBucket.lastRefill;
  const tokensToAdd = Math.floor(elapsed * tokenBucket.refillRate);

  if (tokensToAdd > 0) {
    tokenBucket.tokens = Math.min(tokenBucket.maxTokens, tokenBucket.tokens +
tokensToAdd);
    tokenBucket.lastRefill = now;
  }
}

async function waitForToken(log) {
  while (tokenBucket.tokens < 1) {
    refillTokens();
    if (tokenBucket.tokens < 1) {
      log(`[History][RateLimit] waiting for token, current=$
{tokenBucket.tokens.toFixed(2)}`);
      await new Promise(resolve => setTimeout(resolve, 100));
    }
  }
  tokenBucket.tokens -= 1;
  refillTokens();
}

function sessionIsFresh() {
  return yahooSession.cookie &&
         yahooSession.crumb &&
         (Date.now() - yahooSession.acquiredAt) < SESSION_TTL_MS;
}

function addJitter(delayMs) {
  const jitter = 0.2 * delayMs * (Math.random() - 0.5) * 2;
  return Math.round(delayMs + jitter);
}

async function fetchCrumbWithBackoff(log) {
    const maxAttempts = 6;
    const baseDelayMs = 1000;

    for (let attempt = 1; attempt <= maxAttempts; attempt++) {
        try {
            log(`[History][Stage:crumb] acquiring crumb attempt ${attempt}/$
{maxAttempts}`);
```

```
            await waitForToken(log);

            const cookieResponse = await fetch('https://fc.yahoo.com/', {
                headers: {
                    'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36'
                }
            });

            if (cookieResponse.status === 429 || cookieResponse.status === 999
|| cookieResponse.status >= 500) {
                throw new Error(`Cookie fetch failed: status=$
{cookieResponse.status}`);
            }

            const cookieHeader = cookieResponse.headers.get('set-cookie');
            if (!cookieHeader) {
                throw new Error("No session cookie received from fc.yahoo.com");
            }

            await waitForToken(log);
            const crumbResponse = await
fetch('https://query1.finance.yahoo.com/v1/test/getcrumb', {
                headers: {
                    'cookie': cookieHeader,
                    'User-Agent': 'Mozilla/5.0 (Macintosh; Intel OS X 10_15_7)
AppleWebKit/537.36'
                }
            });

            if (crumbResponse.status === 429 || crumbResponse.status === 999 ||
crumbResponse.status >= 500) {
                throw new Error(`Crumb fetch failed: status=$
{crumbResponse.status}`);
            }

            if (!crumbResponse.ok) {
                throw new Error(`Crumb fetch failed: status=$
{crumbResponse.status}`);
            }

            const crumb = await crumbResponse.text();
            if (!crumb || crumb.length < 5) {
                throw new Error("Invalid crumb received");
            }

            yahooSession.cookie = cookieHeader;
            yahooSession.crumb = crumb;
            yahooSession.acquiredAt = Date.now();

            log("[History][Stage:session] cookie OK");
            log("[History][Stage:session] crumb OK");
            return;

        } catch (error) {
            const isRetriable = error.message.includes('status=429') ||
                                error.message.includes('status=999') ||
                                error.message.includes('status=5') ||
                                error.message.includes('Invalid crumb') ||
                                error.message.includes('No session cookie');

            if (!isRetriable || attempt === maxAttempts) {
                throw {
                    stage: 'session',
```

```
                    code: 'session_error',
                    message: 'Session bootstrap failed after retries',
                    cause: error.message.slice(0, 120)
                };
            }

            const baseDelay = baseDelayMs * Math.pow(1.5, attempt - 1);
            const delay = addJitter(baseDelay);

            log(`[History][Retry] stage=session attempt=${attempt}/$
{maxAttempts} → backing off ${(delay/1000).toFixed(1)}s`);
            await new Promise(resolve => setTimeout(resolve, delay));
        }
    }
}

async function ensureSession(log) {
    log("[History][Stage:session] getSession start");

    if (sessionIsFresh()) {
        log("[History][Stage:session] cache hit");
        return;
    }

    log("[History][Stage:session] cache miss");

    if (yahooSession.inFlight) {
        log("[History][Stage:session] waiting for in-flight session
acquisition");
        await yahooSession.inFlight;
        return;
    }

    yahooSession.inFlight = fetchCrumbWithBackoff(log);

    try {
        await yahooSession.inFlight;
    } finally {
        yahooSession.inFlight = null;
    }
}

async function signedFetch(path, params, log) {
    await ensureSession(log);

    const maxAttempts = 3;
    let lastError = null;

    for (let attempt = 1; attempt <= maxAttempts; attempt++) {
        try {
            const url = new URL(`https://query1.finance.yahoo.com${path}`);
            const finalParams = { ...params, crumb: yahooSession.crumb };
            Object.entries(finalParams).forEach(([key, value]) =>
url.searchParams.append(key, value));

            await waitForToken(log);

            const response = await fetch(url.toString(), {
                headers: {
                    'cookie': yahooSession.cookie,
                    'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36'
                }
            });
```

```
            const bodyText = await response.text();
            log(`[History][Stage:fetch] <- ${path} status=${response.status}
len=${bodyText.length}`);

            if (response.status === 401 || response.status === 403) {
                if (attempt < maxAttempts) {
                    log(`[History][Stage:session] session invalid, invalidating
cache for retry`);
                    yahooSession.cookie = null;
                    yahooSession.crumb = null;
                    yahooSession.acquiredAt = 0;
                    await ensureSession(log);
                    continue;
                }
            }

            if (response.status === 429 || response.status === 999 ||
response.status >= 500) {
                if (attempt < maxAttempts) {
                    const baseDelay = 700 * Math.pow(2, attempt - 1);
                    const delay = addJitter(baseDelay);
                    log(`[History][Retry] stage=fetch attempt=${attempt}/$
{maxAttempts} code=${response.status} → backing off $
{(delay/1000).toFixed(1)}s`);
                    await new Promise(res => setTimeout(res, delay));
                    continue;
                }
            }

            if (!response.ok) {
                throw new Error(`Yahoo fetch failed with status $
{response.status}`);
            }

            try {
                const json = JSON.parse(bodyText);
                return { ok: true, json, status: response.status };
            } catch (parseError) {
                return { ok: false, parseError: true, status: response.status,
text: bodyText };
            }

        } catch (error) {
            lastError = error.message;
            if (attempt < maxAttempts) {
                const delay = addJitter(1000 * attempt);
                log(`[History][Retry] stage=fetch attempt=${attempt}/$
{maxAttempts} code=network_error → backing off ${(delay/1000).toFixed(1)}s`);
                await new Promise(res => setTimeout(res, delay));
                continue;
            }
        }
    }

    throw {
        stage: 'fetch',
        code: 'fetch_failed',
        message: 'Network error after retries',
        cause: lastError?.slice(0, 120) || 'unknown'
    };
}

// --- DATA NORMALIZATION ---
```

```
const toYMD = (d) => d ? new Date(d).toISOString().slice(0, 10) : null;
const numOrNull = (v) => (v === null || v === undefined || Number.isNaN(+v)) ?
null : +v;

function normalizePrices(ticker, priceData, log) {
    log(`[History][Stage:parse] normalizing 5-year price data`);
    const asOfDate = toYMD(new Date());
    const timestamps = Array.isArray(priceData?.timestamp) ? priceData.timestamp
: [];
    if (timestamps.length === 0) return [];

    log(`[History][Stage:parse] processing ${timestamps.length} price timestamps
for 5-year history`);

    const quotes = priceData.indicators?.quote?.[0];
    const adjCloses = priceData.indicators?.adjclose?.[0]?.adjclose;

    if (!quotes || !adjCloses) {
        log(`[History][Stage:parse] missing quote or adjclose data in Yahoo
response`);
        return [];
    }

    const normalized = timestamps.map((ts, i) => ({
        ticker,
        date: toYMD(new Date(ts * 1000)),
        open: numOrNull(quotes.open?.[i]),
        high: numOrNull(quotes.high?.[i]),
        low: numOrNull(quotes.low?.[i]),
        close: numOrNull(quotes.close?.[i]),        // Split-adjusted only
(Yahoo "Close")
        close_adj: numOrNull(adjCloses[i]),         // Dividend & split-adjusted
(Yahoo "Adj Close")
        volume: numOrNull(quotes.volume?.[i]),
        source: 'yahoo',
        as_of_date: asOfDate,
    })).filter(p => p.date && p.close_adj !== null && p.close !== null); //
Require both prices

    log(`[History][Stage:parse] normalized ${normalized.length} valid price
points over 5 years`);
    return normalized;
}

// --- DATABASE UPSERT WITH BATCHING ---
async function upsertHistoricalPrices(base44, prices, log) {
    const safePrices = Array.isArray(prices) ? prices : [];
    if (safePrices.length === 0) {
        log("[History][Stage:db] No historical price data to upsert.");
        return { inserted: 0, updated: 0, skipped: 0 };
    }

    log("[History][Stage:db] Starting 5-year historical price upsert with
service role...");

    const serviceClient = base44.asServiceRole;

    try {
        const testQuery = await serviceClient.entities.PricesHistory.filter({},
null, 1);
        log(`[History][Stage:db] Database connectivity test: ${testQuery ?
'OK' : 'FAILED'}`);
    } catch (dbTestError) {
```

```
        log(`[History][Stage:db] Service role test failed: $
{dbTestError.message}`);
        throw {
            stage: 'db',
            code: 'SERVICE_ROLE_ACCESS_FAILED',
            message: 'Service role database access failed during connectivity
test',
            cause: dbTestError.message
        };
    }

    const ticker = safePrices[0].ticker;
    const BATCH_SIZE = 50; // Larger batches for historical data
    const INTER_BATCH_DELAY = 100; // Shorter delay for bulk operations

    const batches = [];
    for (let i = 0; i < safePrices.length; i += BATCH_SIZE) {
        batches.push(safePrices.slice(i, i + BATCH_SIZE));
    }
    log(`[History][Stage:db] Processing ${batches.length} batches of max $
{BATCH_SIZE} rows each for 5-year history`);

    let totalInserted = 0, totalUpdated = 0, totalSkipped = 0;

    for (let i = 0; i < batches.length; i++) {
        const batch = batches[i];
        const dates = batch.map(p => p.date);

        try {
            const existing = await serviceClient.entities.PricesHistory.filter({
                ticker,
                date: { '$in': dates }
            });

            const safeExisting = Array.isArray(existing) ? existing : [];
            const existingMap = new Map(safeExisting.map(p => [p.date, p]));
            const toInsert = batch.filter(row => !existingMap.has(row.date));
            const toUpdate = batch.filter(row => existingMap.has(row.date));

            log(`[History][Stage:db] Batch ${i+1}: ${toInsert.length} to insert,
${toUpdate.length} to update`);

            if (toInsert.length > 0) {
                const maxRetries = 3;
                const retryDelays = [500, 1000, 2000];

                for (let attempt = 0; attempt < maxRetries; attempt++) {
                    try {
                        await
serviceClient.entities.PricesHistory.bulkCreate(toInsert);
                        totalInserted += toInsert.length;
                        log(`[History][Stage:db] Batch ${i+1} inserts
successful: ${toInsert.length} rows`);
                        break;

                    } catch (dbError) {
                        const status = dbError.response?.status ||
dbError.status || 500;

                        if (status === 401 || status === 403) {
                            log(`[History][Stage:db] φ  UNAUTHORIZED ERROR ($
{status}) on batch ${i+1}`);
                            throw {
                                stage: 'db',
```

```
                                code: 'UNAUTHORIZED',
                                message: `Unauthorized at DB stage (${status});
check service role configuration.`,
                                cause: dbError.message,
                                batch: i + 1,
                                operation: 'bulkCreate'
                            };
                        }

                        if (attempt === maxRetries - 1) {
                            throw {
                                stage: 'db',
                                code: status || 'DB_WRITE_FAILED',
                                message: `DB write for batch ${i+1} failed after
${maxRetries} retries.`,
                                cause: dbError.message
                            };
                        } else {
                            await new Promise(resolve => setTimeout(resolve,
retryDelays[attempt]));
                        }
                    }
                }
            }

            // Handle updates
            for (const row of toUpdate) {
                const existingRow = existingMap.get(row.date);
                if (!existingRow) continue;

                try {
                    await
serviceClient.entities.PricesHistory.update(existingRow.id, row);
                    totalUpdated++;
                } catch (updateError) {
                    log(`[History][Stage:db] Update error for ${row.ticker} $
{row.date}: ${updateError.message}`);
                    totalSkipped++;
                }
            }

        } catch (batchError) {
            if (batchError.stage) {
                throw batchError;
            }

            log(`[History][Stage:db] Batch ${i+1} failed: $
{batchError.message}`);
            totalSkipped += batch.length;
        }

        // Inter-batch delay
        if (i < batches.length - 1) {
            await new Promise(resolve => setTimeout(resolve,
INTER_BATCH_DELAY));
        }
    }

    log(`[History][Stage:db] 5-year history complete. Total inserted: $
{totalInserted}, updated: ${totalUpdated}, skipped: ${totalSkipped}`);
    return { inserted: totalInserted, updated: totalUpdated, skipped:
totalSkipped };
}
```

```
// --- MAIN HANDLER ---
Deno.serve(async (req) => {
    const base44 = createClientFromRequest(req);
    const logs = [];
    const log = (msg) => logs.push(msg);

    let requestTicker = null;

    try {
        log(`[History][Stage:init] starting 5-year historical price sync`);

        if (!(await base44.auth.isAuthenticated())) {
            throw { stage: 'auth', code: 'unauthorized', message:
'Authentication required' };
        }

        const body = await req.json();
        requestTicker = body.ticker;
        const { ticker } = body;

        if (!ticker || typeof ticker !== 'string' || !/^[A-Z.-]+$/.test(ticker))
{
            throw { stage: 'validation', code: 'invalid_ticker', message:
'Invalid or missing ticker symbol.' };
        }

        log("[History] Using YahooSession for 5-year backfill");

        // Fetch 5 years of historical data instead of 1 month
        const urlPath = `/v8/finance/chart/${ticker}`;
        const params = {
            interval: '1d',
            events: 'div%2Csplits',
            range: '5y', // 5 years of data
            includeAdjustedClose: 'true'
        };

        const yahooFetchResult = await signedFetch(urlPath, params, log);

        if (!yahooFetchResult.ok || !yahooFetchResult.json) {
            throw {
                stage: 'fetch',
                code: 'api_error',
                message: `Yahoo API fetch failed with status $
{yahooFetchResult.status}.`,
                cause: yahooFetchResult.text?.slice(0, 120) || 'No response
body'
            };
        }

        const normalized = normalizePrices(ticker,
yahooFetchResult.json.chart.result[0], log);

        if (normalized.length === 0) {
            log("[History] No historical data to sync.");
            return new Response(JSON.stringify({
                ok: true,
                received: 0,
                inserted: 0,
                updated: 0,
                skipped: 0,
                logs
            }), { status: 200, headers: { 'Content-Type':
'application/json' } });
```

```
        }

        const dbOperationResult = await upsertHistoricalPrices(base44,
normalized, log);

        const dates = normalized.map(p => p.date).sort();
        const minDate = dates[0];
        const maxDate = dates[dates.length - 1];

        log(`[History SUCCESS] 5-year window ${minDate} → ${maxDate}, fetched $
{normalized.length}, inserted ${dbOperationResult.inserted}, updated $
{dbOperationResult.updated}, failed ${dbOperationResult.skipped}.`);

        return new Response(JSON.stringify({
            ok: true,
            received: normalized.length,
            inserted: dbOperationResult.inserted,
            updated: dbOperationResult.updated,
            skipped: dbOperationResult.skipped,
            date_range: { min: minDate, max: maxDate },
            logs
        }), {
            status: 200,
            headers: { 'Content-Type': 'application/json' }
        });

    } catch (error) {
        log(`[History ERROR] stage=${error.stage || 'unknown'} code=${error.code
|| 'unknown'} detail=${error.message}`);

        const errorPayload = {
            ok: false,
            stage: error.stage || 'unknown',
            message: error.message,
            code: error.code || 'unknown',
            cause: error.cause || null,
            logs
        };
        return new Response(JSON.stringify(errorPayload), {
            status: error.status || 500,
            headers: { 'Content-Type': 'application/json' }
        });
    }
});




import { createClientFromRequest } from 'npm:@base44/sdk@0.5.0';

// --- START: SHARED CHART REQUEST BUILDER ---

function buildChartRequest(opts) {
    const { symbol, interval = '1d', events = 'div%2Csplits', range, period1,
period2 } = opts;

    if (!symbol) {
        throw new Error('Symbol is required for chart request');
    }

    const urlPath = `/v8/finance/chart/${symbol}`;
    const params = {
        interval,
```

```
        events, // Pre-encoded
        includeAdjustedClose: 'true'
    };

    // Time window: use either range OR period1/period2, never both
    if (period1 && period2) {
        params.period1 = period1.toString();
        params.period2 = period2.toString();
    } else if (range) {
        params.range = range;
    } else {
        throw new Error('Either range or period1/period2 must be specified');
    }

    // Build query string (already encoded once)
    const urlQuery = new URLSearchParams(params).toString();

    return { urlPath, urlQuery, params };
}

async function chartProbe(opts, log) {
    const { urlPath, urlQuery, params } = buildChartRequest(opts);

    try {
        await ensureSession(log);

        const url = new URL(`https://query1.finance.yahoo.com${urlPath}`);
        const finalParams = { ...params, crumb: yahooSession.crumb };
        Object.entries(finalParams).forEach(([key, value]) =>
url.searchParams.append(key, value));

        await waitForToken(log);

        const response = await fetch(url.toString(), {
            headers: {
                'cookie': yahooSession.cookie,
                'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36'
            }
        });

        const bodyText = await response.text();
        const contentType = response.headers.get('content-type') || 'unknown';
        let parsedOk = false;

        if (response.status !== 200) {
            log(`[ChartProbe] status=${response.status} path=${urlPath}
query=<safe: ${urlQuery}>`);
        }

        try {
            JSON.parse(bodyText);
            parsedOk = true;
        } catch {
            parsedOk = false;
        }

        return {
            status: response.status,
            contentType,
            len: bodyText.length,
            parsedOk,
            body: bodyText
        };
```

```
    } catch (error) {
        log(`[ChartProbe] ERROR: ${error.message}`);
        return {
            status: 0,
            contentType: 'error',
            len: 0,
            parsedOk: false,
            error: error.message
        };
    }
}

// --- END: SHARED CHART REQUEST BUILDER ---

// --- START: HARDENED YAHOO SESSION MANAGEMENT ---

// Module-level session cache with TTL and single-flight protection
let yahooSession = {
  cookie: null,
  crumb: null,
  acquiredAt: 0,
  inFlight: null // Promise to prevent concurrent crumb fetches
};

const SESSION_TTL_MS = 60 * 60 * 1000; // 60 minutes
const YAHOO_THROTTLE_MS = parseInt(Deno.env.get("YAHOO_THROTTLE_MS") || "1200");

// Concurrency guard for function entry
let activeRuns = new Map(); // ticker -> promise

// Token bucket rate limiter
let tokenBucket = {
  tokens: 2, // burst capacity
  lastRefill: Date.now(),
  refillRate: 1 / 600, // 1 token per 600ms
  maxTokens: 2
};

function refillTokens() {
  const now = Date.now();
  const elapsed = now - tokenBucket.lastRefill;
  const tokensToAdd = Math.floor(elapsed * tokenBucket.refillRate);

  if (tokensToAdd > 0) {
    tokenBucket.tokens = Math.min(tokenBucket.maxTokens, tokenBucket.tokens +
tokensToAdd);
    tokenBucket.lastRefill = now;
  }
}

async function waitForToken(log) {
  while (tokenBucket.tokens < 1) {
    refillTokens();
    if (tokenBucket.tokens < 1) {
      log(`[Recent][RateLimit] waiting for token, current=$
{tokenBucket.tokens.toFixed(2)}`);
      await new Promise(resolve => setTimeout(resolve, 100));
    }
  }
  tokenBucket.tokens -= 1;
  refillTokens(); // Update after consumption
}
```

```
function sessionIsFresh() {
  return yahooSession.cookie &&
         yahooSession.crumb &&
         (Date.now() - yahooSession.acquiredAt) < SESSION_TTL_MS;
}

// Add jitter to delay (±20%)
function addJitter(delayMs) {
  const jitter = 0.2 * delayMs * (Math.random() - 0.5) * 2;
  return Math.round(delayMs + jitter);
}

async function parseJsonOrHtml(response, log) {
    const text = await response.text();
    if (text.startsWith('<') && (text.includes('<!DOCTYPE html>') ||
text.includes('<html'))) {
        log(`[YahooFetch] Non-JSON (HTML) status=${response.status}`);
        return { ok: false, isHtml: true, status: response.status, text };
    }
    try {
        const json = JSON.parse(text);
        log(`[YahooFetch] json OK`);
        return { ok: true, isJson: true, json, status: response.status };
    } catch (e) {
        log(`[YahooFetch] Non-JSON response detected (Parse Error). status=$
{response.status}`);
        return { ok: false, parseError: true, status: response.status, text };
    }
}

async function fetchCrumbWithBackoff(log) {
    const maxAttempts = 6;
    const baseDelayMs = 1000;

    for (let attempt = 1; attempt <= maxAttempts; attempt++) {
        try {
            log(`[Recent][Stage:crumb] acquiring crumb attempt ${attempt}/$
{maxAttempts}`);

            // Token bucket rate limiting
            await waitForToken(log);

            // Step A: Get Cookie from stable Yahoo Finance page
            const cookieResponse = await fetch('https://fc.yahoo.com/', {
                headers: {
                    'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36'
                }
            });

            if (cookieResponse.status === 429 || cookieResponse.status === 999
|| cookieResponse.status >= 500) {
                throw new Error(`Cookie fetch failed: status=$
{cookieResponse.status}`);
            }

            const cookieHeader = cookieResponse.headers.get('set-cookie');
            if (!cookieHeader) {
                throw new Error("No session cookie received from fc.yahoo.com");
            }

            // Step B: Get Crumb
            await waitForToken(log);
```

```
            const crumbResponse = await
fetch('https://query1.finance.yahoo.com/v1/test/getcrumb', {
                headers: {
                    'cookie': cookieHeader,
                    'User-Agent': 'Mozilla/5.0 (Macintosh; Intel OS X 10_15_7)
AppleWebKit/537.36'
                }
            });

            if (crumbResponse.status === 429 || crumbResponse.status === 999 ||
crumbResponse.status >= 500) {
                throw new Error(`Crumb fetch failed: status=$
{crumbResponse.status}`);
            }

            if (!crumbResponse.ok) {
                throw new Error(`Crumb fetch failed: status=$
{crumbResponse.status}`);
            }

            const crumb = await crumbResponse.text();
            if (!crumb || crumb.length < 5) {
                throw new Error("Invalid crumb received");
            }

            // Success - update session
            yahooSession.cookie = cookieHeader;
            yahooSession.crumb = crumb;
            yahooSession.acquiredAt = Date.now();

            log("[Recent][Stage:session] cookie OK");
            log("[Recent][Stage:session] crumb OK");
            return;

        } catch (error) {
            const isRetriable = error.message.includes('status=429') ||
                                error.message.includes('status=999') ||
                                error.message.includes('status=5') ||
                                error.message.includes('Invalid crumb') ||
                                error.message.includes('No session cookie');

            if (!isRetriable || attempt === maxAttempts) {
                const code = error.message.includes('429') ||
error.message.includes('999') ? 'rate_limited' :
                            error.message.includes('5') ? 'server_error' :
'session_error';
                throw {
                    stage: 'session',
                    code: code,
                    message: 'Session bootstrap failed after retries',
                    cause: error.message.slice(0, 120)
                };
            }

            // Calculate backoff with jitter
            const baseDelay = baseDelayMs * Math.pow(1.5, attempt - 1);
            const delay = addJitter(baseDelay);

            log(`[Recent][Retry] stage=session attempt=${attempt}/${maxAttempts}
code=${error.message.includes('429') || error.message.includes('999') ? '429' :
'5xx'} → backing off ${(delay/1000).toFixed(1)}s`);
            await new Promise(resolve => setTimeout(resolve, delay));
        }
    }
```

```
}

async function ensureSession(log) {
    log("[Recent][Stage:session] getSession start");

    // Check if session is still fresh
    if (sessionIsFresh()) {
        log("[Recent][Stage:session] cache hit");
        return;
    }

    log("[Recent][Stage:session] cache miss");

    // If a fetch is already in progress, wait for it
    if (yahooSession.inFlight) {
        log("[Recent][Stage:session] waiting for in-flight session
acquisition");
        await yahooSession.inFlight;
        return;
    }

    // Start new session acquisition
    yahooSession.inFlight = fetchCrumbWithBackoff(log);

    try {
        await yahooSession.inFlight;
    } finally {
        yahooSession.inFlight = null;
    }
}

async function signedFetch(path, params, log) {
    await ensureSession(log);

    const maxAttempts = 3;
    let lastError = null;

    for (let attempt = 1; attempt <= maxAttempts; attempt++) {
        try {
            const url = new URL(`https://query1.finance.yahoo.com${path}`);
            const finalParams = { ...params, crumb: yahooSession.crumb };
            Object.entries(finalParams).forEach(([key, value]) =>
url.searchParams.append(key, value));

            const safeQuery = new URLSearchParams(params).toString();
            log(`[Recent][Stage:fetch] GET ${path}?${safeQuery} (attempt $
{attempt})`);

            // Token bucket rate limiting
            await waitForToken(log);

            const response = await fetch(url.toString(), {
                headers: {
                    'cookie': yahooSession.cookie,
                    'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X
10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/114.0.0.0 Safari/537.36'
                }
            });

            const bodyText = await response.text();
            log(`[Recent][Stage:fetch] <- ${path} status=${response.status}
len=${bodyText.length}`);

            // Handle authentication errors
```

```
            if (response.status === 401 || response.status === 403) {
                if (attempt < maxAttempts) {
                    log(`[Recent][Stage:session] session invalid, invalidating
cache for retry`);
                    yahooSession.cookie = null;
                    yahooSession.crumb = null;
                    yahooSession.acquiredAt = 0;
                    await ensureSession(log);
                    continue;
                }
                throw {
                    stage: 'fetch',
                    code: 'session_error',
                    message: 'Authentication failed after session refresh',
                    cause: `status=${response.status}`
                };
            }

            // Handle rate limiting and server errors
            if (response.status === 429 || response.status === 999 ||
response.status >= 500) {
                if (attempt < maxAttempts) {
                    const baseDelay = 700 * Math.pow(2, attempt - 1); // 700ms,
1400ms, 2800ms
                    const delay = addJitter(baseDelay);
                    log(`[Recent][Retry] stage=fetch attempt=${attempt}/$
{maxAttempts} code=${response.status} → backing off $
{(delay/1000).toFixed(1)}s`);
                    await new Promise(res => setTimeout(res, delay));
                    continue;
                }
                const code = (response.status === 429 || response.status ===
999) ? 'rate_limited' : 'server_error';
                throw {
                    stage: 'fetch',
                    code: code,
                    message: `${code === 'rate_limited' ? 'Rate limited' :
'Server error'} after retries`,
                    cause: `status=${response.status} len=${bodyText.length}`
                };
            }

            // Handle client errors (don't retry)
            if (response.status >= 400 && response.status < 500) {
                throw {
                    stage: 'fetch',
                    code: 'client_error',
                    message: 'Client error from Yahoo API',
                    cause: `status=${response.status} body=${bodyText.slice(0,
120)}`
                };
            }

            // Parse response
            log(`[Recent][Stage:parse] parsing response`);
            const mockResponse = { status: response.status, text: () =>
Promise.resolve(bodyText) };
            const parsed = await parseJsonOrHtml(mockResponse, log);

            if (parsed.isHtml && attempt < maxAttempts) {
                lastError = "Received HTML instead of JSON";
                log(`[Recent][Retry] stage=parse attempt=${attempt}/$
{maxAttempts} code=html_response`);
                continue;
```

```
            }

            if (!parsed.ok) {
                throw {
                    stage: 'parse',
                    code: parsed.isHtml ? 'html_response' : 'parse_error',
                    message: parsed.isHtml ? 'Received HTML instead of JSON' :
'JSON parse error',
                    cause: parsed.text?.slice(0, 120) || 'unknown'
                };
            }

            return parsed;

        } catch (error) {
            if (error.stage) {
                // Structured error, re-throw
                throw error;
            }

            // Network or unexpected error
            lastError = error.message;
            if (attempt < maxAttempts) {
                const delay = addJitter(1000 * attempt);
                log(`[Recent][Retry] stage=fetch attempt=${attempt}/$
{maxAttempts} code=network_error → backing off ${(delay/1000).toFixed(1)}s`);
                await new Promise(res => setTimeout(res, delay));
                continue;
            }
        }
    }

    throw {
        stage: 'fetch',
        code: 'network_error',
        message: 'Network error after retries',
        cause: lastError?.slice(0, 120) || 'unknown'
    };
}

// --- DATA NORMALIZATION & DB HELPERS ---
const toYMD = (d) => d ? new Date(d).toISOString().slice(0, 10) : null;
const numOrNull = (v) => (v === null || v === undefined || Number.isNaN(+v)) ?
null : +v;

function normalizePrices(ticker, priceData, log) {
    log(`[Recent][Stage:parse] normalizing price data`);
    const asOfDate = toYMD(new Date());
    const timestamps = Array.isArray(priceData?.timestamp) ? priceData.timestamp
: [];
    if (timestamps.length === 0) return [];

    log(`[Recent][Stage:parse] processing ${timestamps.length} price
timestamps`);

    const quotes = priceData.indicators?.quote?.[0];
    const adjCloses = priceData.indicators?.adjclose?.[0]?.adjclose;

    if (!quotes || !adjCloses) {
        log(`[Recent][Stage:parse] missing quote or adjclose data in Yahoo
response`);
        return [];
    }
```

```
    const normalized = timestamps.map((ts, i) => ({
        ticker,
        date: toYMD(new Date(ts * 1000)),
        open: numOrNull(quotes.open?.[i]),
        high: numOrNull(quotes.high?.[i]),
        low: numOrNull(quotes.low?.[i]),
        close: numOrNull(quotes.close?.[i]),        // Split-adjusted only
(Yahoo "Close")
        close_adj: numOrNull(adjCloses[i]),         // Dividend & split-adjusted
(Yahoo "Adj Close")
        volume: numOrNull(quotes.volume?.[i]),
        source: 'yahoo',
        as_of_date: asOfDate,
    })).filter(p => p.date && p.close_adj !== null && p.close !== null); //
Require both prices

    log(`[Recent][Stage:parse] normalized ${normalized.length} valid price
points`);
    return normalized;
}

// --- Enhanced DB Write Logic with Comprehensive Auth Diagnostics ---
async function upsertPricesWithRetry(base44, prices, log) {
    const safePrices = Array.isArray(prices) ? prices : [];
    if (safePrices.length === 0) {
        log("[Recent][Stage:db] No price data to upsert.");
        return { inserted: 0, updated: 0, skipped: 0 };
    }

    // AUTH DIAGNOSTICS
    log("[Recent][Stage:db] Starting database operations with comprehensive auth
diagnostics...");

    // Initialize service client early for diagnostics and subsequent use
    const serviceClient = base44.asServiceRole;

    try {
        // Quick connectivity test
        log("[Recent][Stage:db] Using service role client for database
operations");
        const testQuery = await serviceClient.entities.PricesHistory.filter({},
null, 1);
        log(`[Recent][Stage:db] Database connectivity test: ${testQuery ?
'CONNECTED' : 'FAILED'}`);

    } catch (authTestError) {
        log(`[Recent][Stage:db] Service role test failed: $
{authTestError.message}`);
        throw {
            stage: 'db',
            code: 'SERVICE_ROLE_ACCESS_FAILED',
            message: 'Service role database access failed during connectivity
test',
            cause: authTestError.message
        };
    }

    const ticker = safePrices[0].ticker;
    const BATCH_SIZE = 25; // Reduced batch size for better error isolation
    const INTER_BATCH_DELAY = 200; // 200ms between batches

    const batches = [];
    for (let i = 0; i < safePrices.length; i += BATCH_SIZE) {
        batches.push(safePrices.slice(i, i + BATCH_SIZE));
```

```
    }
    log(`[Recent][Stage:db] Processing ${batches.length} batches of max $
{BATCH_SIZE} rows each`);

    let totalInserted = 0, totalUpdated = 0, totalSkipped = 0;

    for (let i = 0; i < batches.length; i++) {
        const batch = batches[i];
        const dates = batch.map(p => p.date);

        log(`[Recent][Stage:db] Processing batch ${i+1}/${batches.length} with $
{batch.length} rows`);

        try {
            // Check for existing records using service role
            const existing = await serviceClient.entities.PricesHistory.filter({
                ticker,
                date: { '$in': dates }
            });

            const safeExisting = Array.isArray(existing) ? existing : [];
            const existingMap = new Map(safeExisting.map(p => [p.date, p]));
            const toInsert = batch.filter(row => !existingMap.has(row.date));
            const toUpdate = batch.filter(row => existingMap.has(row.date));

            log(`[Recent][Stage:db] Batch ${i+1}: ${toInsert.length} to insert,
${toUpdate.length} to update`);

            // Handle inserts
            if (toInsert.length > 0) {
                const maxRetries = 3;
                const retryDelays = [1500, 3000, 5000];

                for (let attempt = 0; attempt < maxRetries; attempt++) {
                    try {
                        await
serviceClient.entities.PricesHistory.bulkCreate(toInsert);
                        totalInserted += toInsert.length;
                        log(`[Recent][Stage:db] Batch ${i+1} inserts successful:
${toInsert.length} rows`);
                        break; // Success

                    } catch (dbError) {
                        const status = dbError.response?.status ||
dbError.status || 500;
                        log(`[Recent][Stage:db] Insert error on batch ${i+1},
attempt ${attempt+1}: status=${status}, message=${dbError.message}`);

                        if (status === 429) {
                            // Rate limit - check Retry-After header
                            const retryAfter =
dbError.response?.headers?.get('retry-after');
                            const delay = retryAfter ? parseInt(retryAfter) *
1000 : retryDelays[attempt];

                            if (attempt < maxRetries - 1) {
                                log(`[Recent][Stage:db] Rate limit (429) on
batch ${i+1} - retrying in ${delay}ms`);
                                await new Promise(resolve => setTimeout(resolve,
delay));
                                continue;
                            }
                        } else if (status === 401 || status === 403) {
```

```
                            // Authentication error - provide detailed
diagnostics
                            log(`[Recent][Stage:db] φ  UNAUTHORIZED ERROR ($
{status}) on batch ${i+1}`);
                            log(`[Recent][Stage:db] φ  Using service role
client: ${!!serviceClient}`);
                            log(`[Recent][Stage:db] φ  Error message: $
{dbError.message}`);
                            log(`[Recent][Stage:db] φ  This indicates RLS policy
or service role configuration issues`);

                            throw {
                                stage: 'db',
                                code: 'UNAUTHORIZED',
                                message: `Unauthorized at DB stage (${status});
check service key / role / RLS policy for PricesHistory table.`,
                                cause: dbError.message,
                                batch: i + 1,
                                operation: 'bulkCreate'
                            };
                        }

                        // Final attempt failed
                        if (attempt === maxRetries - 1) {
                            throw {
                                stage: 'db',
                                code: status || 'DB_WRITE_FAILED',
                                message: `DB write for batch ${i+1} failed after
${maxRetries} retries.`,
                                cause: dbError.message,
                                resumeHint: `Failed at batch ${i+1}/$
{batches.length}. ${totalInserted} rows inserted so far.`
                            };
                        } else {
                            await new Promise(resolve => setTimeout(resolve,
retryDelays[attempt]));
                        }
                    }
                }
            }

            // Handle updates (individual updates for better error isolation)
            for (const row of toUpdate) {
                const existingRow = existingMap.get(row.date);
                if (!existingRow) continue;

                try {
                    await
serviceClient.entities.PricesHistory.update(existingRow.id, row);
                    totalUpdated++;
                } catch (updateError) {
                    log(`[Recent][Stage:db] Update error for ${row.ticker} $
{row.date}: ${updateError.message}`);
                    totalSkipped++;
                }
            }

        } catch (batchError) {
            // If this is already a structured error, re-throw it
            if (batchError.stage) {
                throw batchError;
            }

            // Otherwise, wrap it
```

```
            log(`[Recent][Stage:db] Batch ${i+1} failed: $
{batchError.message}`);
            totalSkipped += batch.length;
        }

        // Inter-batch delay for rate limiting
        if (i < batches.length - 1) {
            await new Promise(resolve => setTimeout(resolve,
INTER_BATCH_DELAY));
        }
    }

    log(`[Recent][Stage:db] All batches complete. Total inserted: $
{totalInserted}, updated: ${totalUpdated}, skipped: ${totalSkipped}`);
    return { inserted: totalInserted, updated: totalUpdated, skipped:
totalSkipped };
}

// --- MAIN HANDLER ---
Deno.serve(async (req) => {
    const base44 = createClientFromRequest(req);
    const logs = [];
    const log = (msg) => logs.push(msg);

    let requestTicker = null; // Declare here to be accessible in finally

    try {
        log(`[Recent][Stage:init] starting recent price sync`);

        if (!(await base44.auth.isAuthenticated())) {
            throw { stage: 'auth', code: 'unauthorized', message:
'Authentication required' };
        }

        const body = await req.json(); // Consume request body once
        requestTicker = body.ticker; // Store ticker
        const { ticker } = body;

        if (!ticker || typeof ticker !== 'string' || !/^[A-Z.-]+$/.test(ticker))
{
            throw { stage: 'validation', code: 'invalid_ticker', message:
'Invalid or missing ticker symbol.' };
        }

        // Concurrency guard
        if (activeRuns.has(ticker)) {
            log(`[Recent][Guard] coalesced concurrent run for ${ticker}`);
            await activeRuns.get(ticker);
            return new Response(JSON.stringify({ ok: true, received: 0,
inserted: 0, updated: 0, skipped: 0, logs }), { status: 200 });
        }

        log(`[Recent][Guard] run locked for ${ticker}`);

        const runPromise = (async () => {
            log("[Recent] Using YahooSession v1");

            // USE SHARED CHART REQUEST BUILDER
            const { urlPath, params } = buildChartRequest({
                symbol: ticker,
                interval: '1d',
                events: 'div%2Csplits',
                range: '1mo'
            });
```

```
            const yahooFetchResult = await signedFetch(urlPath, params, log);

            if (!yahooFetchResult.ok || !yahooFetchResult.isJson) {
                throw { stage: 'fetch', code: 'api_error', message: `Yahoo API
fetch failed with status ${yahooFetchResult.status}.`, cause:
yahooFetchResult.text?.slice(0, 120) || 'No response body' };
            }

            log("[Recent] Using signedFetch + crumb session");

            const normalized = normalizePrices(ticker,
yahooFetchResult.json.chart.result[0], log);

            if (normalized.length === 0) {
                log("[Recent] No new or changed data to sync.");
                return { ok: true, received: 0, inserted: 0, updated: 0,
skipped: 0, logs };
            }

            // Stage: db (use new robust upsert)
            const dbOperationResult = await upsertPricesWithRetry(base44,
normalized, log);

            const dates = normalized.map(p => p.date).sort();
            const minDate = dates[0];
            const maxDate = dates[dates.length - 1];

            log(`[Recent SUCCESS] window ${minDate} → ${maxDate}, fetched $
{normalized.length}, inserted ${dbOperationResult.inserted}, updated $
{dbOperationResult.updated}, failed ${dbOperationResult.skipped}.`);

            return {
                ok: true,
                received: normalized.length,
                inserted: dbOperationResult.inserted,
                updated: dbOperationResult.updated,
                skipped: dbOperationResult.skipped,
                logs
            };
        })();

        activeRuns.set(ticker, runPromise);
        const finalRunResult = await runPromise;
        return new Response(JSON.stringify(finalRunResult), { status: 200 });

    } catch (error) {
        log(`[Recent ERROR] stage=${error.stage || 'unknown'} code=${error.code
|| 'unknown'} detail=${error.message}`);

        const errorPayload = {
            ok: false,
            stage: error.stage || 'unknown',
            message: error.message,
            code: error.code || 'unknown',
            cause: error.cause || null,
            logs
        };
        return new Response(JSON.stringify(errorPayload), { status: error.status
|| 500, headers: { 'Content-Type': 'application/json' }});
    } finally {
        if (requestTicker) { // Use the stored ticker
            activeRuns.delete(requestTicker);
            log(`[Recent][Guard] run unlocked for ${requestTicker}`);
```

```
        }
    }
});




/**
 * Finnhub Fundamentals ETL — ONE DROP-IN FILE
 * Fixes:
 * 1) Typo: PATTERN_SRCS -> PATTERNS (and all references)
 * 2) EBIT pattern reference now PATTERNS.EBIT
 * 3) PE log header clarifies NI/AvgShares
 * 4) Quarter sort is now correct even if Finnhub returns quarter as 1/2/3/4
 * 5) EPS per quarter derived from YTD diffs with proper quarter order (no
spurious negatives)
 * 6) Normalize diluted shares to raw units (auto-detect "in millions")
 * 7) Always overwrite fast-moving metrics for the Projection Tool (eps_ttm,
pe_ttm, pe_12m/24m/36m, current_pe)
 * 8) Ensure Projection Tool field names are written (`pe_12m`, `pe_24m`,
`pe_36m`, `current_pe`)
 * 9) Use split-adjusted close (not dividend-adjusted) for P/E calculations
 */

import { createClientFromRequest } from 'npm:@base44/sdk@0.5.0';

const FALLBACK_KEY = "d2g4fd1r01qkv5nfnjvgd2g4fd1r01qkv5nfnk00";
const FINNHUB_KEY = Deno.env.get("FINNHUB_API_KEY") || FALLBACK_KEY;
const BASE_URL = "https://finnhub.io/api/v1";

// ---------- Rate limit + retry ----------
let lastRequestTime = 0;
const REQUEST_INTERVAL_MS = 1100;
async function gatedFetch(url, log, maxRetries = 3) {
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    const now = Date.now();
    const lag = now - lastRequestTime;
    if (lag < REQUEST_INTERVAL_MS) await new Promise(r => setTimeout(r,
REQUEST_INTERVAL_MS - lag));
    lastRequestTime = Date.now();

    try {
      const res = await fetch(url);
      const remaining = res.headers.get('x-ratelimit-remaining');
      const reset = res.headers.get('x-ratelimit-reset');
      if (remaining) log(`[Finnhub][RateLimit] Remaining: ${remaining}, Reset: $
{reset ? new Date(parseInt(reset) * 1000).toLocaleTimeString() : 'N/A'}`);

      if (res.status === 429) {
        const backoff = Math.pow(2, attempt) * 1000 + Math.random() * 1000;
        log(`[Finnhub][429] backing off ${backoff.toFixed(0)}ms (attempt $
{attempt}/${maxRetries})`);
        if (attempt < maxRetries) { await new Promise(r => setTimeout(r,
backoff)); continue; }
        throw new Error(`Rate limited after ${maxRetries} attempts`);
      }
      if (!res.ok) throw new Error(`HTTP ${res.status}: ${res.statusText}`);
      return await res.json();
    } catch (e) {
      log(`[Finnhub] Attempt ${attempt}/${maxRetries} failed: ${e.message}`);
      if (attempt === maxRetries) throw e;
    }
  }
```

```
}

// ---------- Helpers ----------
const num = v => (typeof v === 'number' && Number.isFinite(v)) ? v
  : (typeof v === 'string' && v.trim() !== '' && !isNaN(+v)) ? +v : null;

function val(arr, patterns) {
  if (!Array.isArray(arr)) return null;
  for (const pattern of patterns) {
    for (const item of arr) {
      if (!item || typeof item !== 'object') continue;
      const concept = String(item.concept || '').toLowerCase();
      const label = String(item.label || '').toLowerCase();
      if (pattern.test(concept) || pattern.test(label)) {
        const n = num(item.value);
        if (n !== null) return n;
      }
    }
  }
  return null;
}

function normalizeShares(v) {
  if (v == null) return null;
  if (v >= 1e8) return v;          // already raw shares
  if (v > 0 && v < 1e6) return v * 1e6; // likely "in millions"
  return v;
}

// Quarter label normalizer: accepts "Q1"/"1"/1 -> "Q1"
const toQ = q => {
  const s = String(q).toUpperCase().replace('QUARTER', '').trim();
  const n = /^[Q]?([1-4])$/.exec(s)?.[1];
  return n ? `Q${n}` : (['Q1','Q2','Q3','Q4'].includes(s) ? s : 'Q1');
};
const qNum = q => ({Q1:1,Q2:2,Q3:3,Q4:4}[toQ(q)] || 1);

// ---------- Pattern sets ----------
const PATTERNS = {
  CFO: [/us-gaap_netcashprovidedbyusedinoperatingactivities/i,
/operating.*cash.*flow/, /net.*cash.*provided.*operating/],
  CapEx: [
    /paymentstoacquirepropertyplantandequipment/i,
    /paymentstoacquireproductiveassets/i,
    /acquisitionofpropertyplantandequipment/i,
    /(acquire|purchase).*property.*plant.*equipment/i,
    /productiveassets/i,
    /capital.*expend/i
  ],
  SBC: [/us-gaap_sharebasedcompensation/i, /share.*based.*comp/,
/stock.*based.*comp/],
  Depreciation: [/us-gaap_depreciationdepletionandamortization/i,
/deprecia(tion)?(?!.*tax)/, /deprecia.*amor[tz]/],
  InterestExp: [/interest.*expense/, /interest.*paid/],
  Cash: [/cash(?!.*flow)/, /cash.*equivalents/, /cash.*short.*term.*invest/i],
  ShortDebt: [/commercial.*paper/i, /short.*term.*debt/,
/current.*portion.*long.*term.*debt/],
  LongDebt: [/longtermdebtnoncurrent/i, /long.*term.*debt(?!.*current)/],
  TotalDebt: [/total.*debt/, /total.*interest.*bearing.*debt/i],
  Equity: [/total.*share.*holder.*equity/, /totalstockholdersequity/i],
  TotalAssets: [/total.*assets/],
  EBIT: [/us-gaap_earningsbeforeinterestandtaxes/i, /us-
gaap_operatingincomeloss/i, /^ebit$/, /earnings.*before.*interest.*tax/],
  NetIncome: [/us-gaap_netincomeloss/i, /^net.*income/],
```

```
  SharesDiluted: [/us-gaap_weightedaveragenumberofdilutedshare/i,
/weighted.*average.*shares.*diluted/],
};

// ---------- Fetch ----------
const fetchQuarterlyFinancials = (ticker, log) =>
  gatedFetch(`${BASE_URL}/stock/financials-reported?symbol=$
{ticker}&freq=quarterly&token=${FINNHUB_KEY}`, log);

const fetchAnnualFinancials = async (ticker, log) => {
  try { return await gatedFetch(`${BASE_URL}/stock/financials-reported?symbol=$
{ticker}&freq=annual&token=${FINNHUB_KEY}`, log); }
  catch (e) { log(`[Finnhub] Annual data fetch failed: ${e.message}`); return
null; }
};

// ---------- Normalize + Quarterize ----------
function normalizeAndQuarterizeData(quarterlyData, annualData, log) {
  const reports = [];
  if (quarterlyData?.data) {
    for (const r of quarterlyData.data) {
      if (!r.report) continue;
      const fy = r.year || new Date(r.endDate || r.acceptedDate ||
'').getFullYear();
      const quarter = toQ(r.quarter || r.fiscalQuarter || r.report?.fp ||
r?.period || 'Q1');
      const endDate = r.endDate || r.acceptedDate || r.period;
      if (!endDate) continue;
      const ic = r.report.ic || [], bs = r.report.bs || [], cf = r.report.cf ||
[];
      reports.push({ type: 'quarterly', fiscalYear: fy, quarter, endDate,
periodEnd: new Date(endDate).toISOString().slice(0,10), ic, bs, cf, report:
r.report });
    }
  }

  const annual = {};
  if (annualData?.data) {
    for (const r of annualData.data) {
      if (!r.report) continue;
      const fy = r.year || new Date(r.endDate || r.acceptedDate ||
'').getFullYear();
      annual[fy] = { ic: r.report.ic || [], cf: r.report.cf || [] };
    }
  }

  // Sort by FY desc then quarter desc for preview logs
  reports.sort((a,b) => (b.fiscalYear - a.fiscalYear) || (qNum(b.quarter) -
qNum(a.quarter)));
  if (reports.length) {
    const f = reports[0];
    log(`[Finnhub][IC] concepts=${f.ic.map(x=>x.concept).join(', ')}`);
    log(`[Finnhub][CF] concepts=${f.cf.map(x=>x.concept).join(', ')}`);
    log(`[Finnhub][BS] concepts=${f.bs.map(x=>x.concept).join(', ')}`);
  }

  const quarterized = [];
  const years = [...new Set(reports.map(r => r.fiscalYear))];

  for (const fy of years) {
    // For differencing, process in true chronological order Q1→Q4
    const yr = reports.filter(r => r.fiscalYear === fy).sort((a,b) =>
qNum(a.quarter) - qNum(b.quarter));
    if (!yr.length) continue;
```

```
    log(`[Quarterize] Processing FY ${fy} with ${yr.length} quarters`);
    const ytdFlows = {};
    const annualFlows = annual[fy] || { ic:[], cf:[] };
    const out = [];

    for (const r of yr) {
      const sharesRaw = val(r.ic, PATTERNS.SharesDiluted);
      const shares_diluted = normalizeShares(sharesRaw);

      const qd = {
        periodEnd: r.periodEnd, fiscalYear: fy, quarter: toQ(r.quarter),
        cfo:null, capex:null, sbc:null, depreciation:null, interest_exp:null,
        ebit:null, net_income:null, diluted_eps:null, shares_diluted,
        cash: val(r.bs, PATTERNS.Cash),
        short_debt: val(r.bs, PATTERNS.ShortDebt),
        long_debt: val(r.bs, PATTERNS.LongDebt),
        total_debt: val(r.bs, PATTERNS.TotalDebt),
        equity: val(r.bs, PATTERNS.Equity),
        total_assets: val(r.bs, PATTERNS.TotalAssets),
      };

      const flowDefs = [
        { key:'cfo', patterns:PATTERNS.CFO, sections:[r.cf] },
        { key:'capex', patterns:PATTERNS.CapEx, sections:[r.cf] },
        { key:'sbc', patterns:PATTERNS.SBC, sections:[r.cf] },
        { key:'depreciation', patterns:PATTERNS.Depreciation, sections:[r.cf] },
        { key:'interest_exp', patterns:PATTERNS.InterestExp, sections:[r.ic] },
        { key:'ebit', patterns:PATTERNS.EBIT, sections:[r.ic] },
        { key:'net_income', patterns:PATTERNS.NetIncome, sections:[r.ic] },
      ];

      for (const d of flowDefs) {
        let ytd = null;
        for (const sec of d.sections) { ytd = val(sec, d.patterns); if (ytd !=
null) break; }
        if (ytd != null) {
          const prev = ytdFlows[d.key] ?? 0;
          log(`[Quarterize][Probe] FY ${fy} ${qNum(r.quarter)} ${d.key}_YTD=$
{ytd} prev_YTD=${prev}`);
          qd[d.key] = (qNum(r.quarter) === 1) ? ytd : (ytd - prev);
          ytdFlows[d.key] = ytd;
        }
      }

      if (qd.net_income != null && shares_diluted && shares_diluted > 0) {
        qd.diluted_eps = qd.net_income / shares_diluted;
        log(`[EPS-Qtr][Calc] FY ${fy} ${toQ(r.quarter)} eps=${qd.diluted_eps}`);
      }
      out.push(qd);
    }

    // Synthesize Q4 if we have only 3 quarters and an annual
    if (out.length === 3 && (annualFlows.ic?.length || annualFlows.cf?.length))
{
      const annualVal = (p) => val([...(annualFlows.ic||[]), ...
(annualFlows.cf||[])], p);
      const y3 = { ...out.find(x => x.quarter === 'Q3') };
      if (y3) {
        const sQ4 = { ...y3, quarter:'Q4' };
        const defs = [
          ['cfo', PATTERNS.CFO],
          ['capex', PATTERNS.CapEx],
          ['sbc', PATTERNS.SBC],
```

```
          ['depreciation', PATTERNS.Depreciation],
          ['interest_exp', PATTERNS.InterestExp],
          ['ebit', PATTERNS.EBIT],
          ['net_income', PATTERNS.NetIncome],
        ];
        for (const [k, pats] of defs) {
          const a = annualVal(pats);
          const y = ytdFlows[k];
          sQ4[k] = (Number.isFinite(a) && Number.isFinite(y)) ? (a - y) : null;
          log(`[Q4] fy=${fy} ${k}: annual=${a} ytd_q3=${y} q4=${sQ4[k]}`);
        }
        if (sQ4.net_income != null && sQ4.shares_diluted && sQ4.shares_diluted >
0) {
          sQ4.diluted_eps = sQ4.net_income / sQ4.shares_diluted;
          log(`[EPS-Qtr][Calc] FY ${fy} Q4 eps=${sQ4.diluted_eps}`);
        } else sQ4.diluted_eps = null;

        out.push(sQ4);
        out.sort((a,b)=> qNum(a.quarter) - qNum(b.quarter));
      }
    }

    quarterized.push(...out);
    log(`[Quarterize] Flows differenced for FY ${fy}`);
  }

  quarterized.sort((a,b) => new Date(b.periodEnd).getTime() - new
Date(a.periodEnd).getTime());
  log(`[Finnhub] Quarterized ${quarterized.length} reports`);
  return quarterized;
}

// ---------- TTM ----------
function buildTTMMetrics(quarters, log) {
  const qAsc = quarters.slice().sort((a,b)=> new Date(a.periodEnd).getTime()-new
Date(b.periodEnd).getTime());
  const last4 = qAsc.slice(-4);
  if (last4.length < 4) { log('[TTM] Not enough quarters for TTM calculation.');
return {}; }

  log(`[TTM][Quarters] ${last4.map(q=>q.periodEnd).join(', ')}`);

  const sum = (f) => last4.some(q=>q[f]==null) ? null :
last4.reduce((s,q)=>s+q[f],0);
  const avg = (f) => last4.some(q=>q[f]==null) ? null :
last4.reduce((s,q)=>s+q[f],0)/4;
  const latest = last4[3];

  const ebit_ttm = sum('ebit');
  const depreciation_ttm = sum('depreciation');
  const ebitda_ttm = (ebit_ttm!=null && depreciation_ttm!=null) ? ebit_ttm +
depreciation_ttm : null;

  const net_income_ttm = sum('net_income');
  const cfo_ttm = sum('cfo');
  const capex_ttm = sum('capex');
  const sbc_ttm = sum('sbc');
  const interest_expense_ttm = sum('interest_exp');

  let eps_ttm = null;
  const shAvg = avg('shares_diluted');
  if (net_income_ttm != null && shAvg && shAvg > 0) eps_ttm = net_income_ttm /
shAvg;
```

```
  const total_debt_calc = (latest.short_debt || 0) + (latest.long_debt || 0);

  const out = {
    cfo_ttm, capex_ttm, sbc_ttm, depreciation_ttm, ebit_ttm, ebitda_ttm,
    net_income_ttm, interest_expense_ttm, eps_ttm,
    cash: latest.cash,
    total_debt: latest.total_debt || (total_debt_calc > 0 ? total_debt_calc :
null),
    equity: latest.equity,
    total_assets: latest.total_assets,
  };
  log(`[TTM] ni=${net_income_ttm} cfo=${cfo_ttm} capex=${capex_ttm} eps_ttm=$
{eps_ttm?.toFixed(4)}`);
  return out;
}

// ---------- PE history (DB prices + NI/AvgShares) ----------
async function calculateHistoricalPE_DB(base44, ticker, quarters, log) {
  log(`[PE-Hist] Calculating with source: prices=DB (Close),
eps_ttm=NI/AvgShares`);

  // Fetch 5 years of price history to ensure robust month-end coverage
  const fiveYearsAgo = new Date();
  fiveYearsAgo.setFullYear(fiveYearsAgo.getFullYear() - 5);

  const priceHistory = await
base44.asServiceRole.entities.PricesHistory.filter({
    ticker,
    date: { $gte: fiveYearsAgo.toISOString().slice(0, 10) }
  }, '-date');

  if (!priceHistory?.length) {
    log(`[PE-Hist] No price history found for ${ticker}. Consider running 5-year
price backfill.`);
    return { pe_ttm_12m_ago: null, pe_ttm_24m_ago: null, pe_ttm_36m_ago: null };
  }

  log(`[PE-Hist] Found ${priceHistory.length} price records for ${ticker}`);

  // Helper function for robust month-end selection
  function pickMonthEnd(priceRows, targetMonth) {
    const start = new Date(targetMonth.getFullYear(), targetMonth.getMonth(),
1);
    const end = new Date(targetMonth.getFullYear(), targetMonth.getMonth() + 1,
0);

    const inMonth = priceRows.filter(row => {
      const rowDate = new Date(row.date);
      return rowDate >= start && rowDate <= end;
    });

    if (inMonth.length === 0) return null;

    // Return the row with the latest date (last trading day of the month)
    return inMonth.reduce((latest, current) => {
      const latestDate = new Date(latest.date);
      const currentDate = new Date(current.date);
      return currentDate > latestDate ? current : latest;
    });
  }

  // Validate price coverage
  function validatePriceCoverage(priceRows, minMonthsToCheck) {
    const coverage = {
```

```
      hasMinimumCoverage: true,
      oldestDate: null,
      newestDate: null,
      totalRows: priceRows.length,
      missingMonths: [],
      recommendation: ''
    };

    if (priceRows.length === 0) {
      coverage.hasMinimumCoverage = false;
      coverage.recommendation = "No price history found. Run 5-year price
backfill.";
      return coverage;
    }

    coverage.newestDate = priceRows[0]?.date;
    coverage.oldestDate = priceRows[priceRows.length - 1]?.date;

    // Check for critical 12, 24, 36 month points
    const targetMonthsAgo = [12, 24, 36];
    for (const monthsAgo of targetMonthsAgo) {
      const targetDate = new Date();
      targetDate.setMonth(targetDate.getMonth() - monthsAgo);
      const monthEndPrice = pickMonthEnd(priceRows, targetDate);
      if (!monthEndPrice) {
        coverage.missingMonths.push(monthsAgo);
        coverage.hasMinimumCoverage = false;
      }
    }

    if (!coverage.hasMinimumCoverage) {
      coverage.recommendation = "Missing price data for key historical periods.
Consider running 5-year price backfill to ensure full coverage.";
    } else {
      coverage.recommendation = "Price history appears sufficient for P/E
calculations.";
    }

    return coverage;
  }

  const coverage = validatePriceCoverage(priceHistory, 4);
  log(`[PE-Hist] Price coverage: ${coverage.hasMinimumCoverage ? 'SUFFICIENT' :
'INSUFFICIENT'}`);
  log(`[PE-Hist] Date range: ${coverage.oldestDate} to ${coverage.newestDate} ($
{coverage.totalRows} rows)`);

  if (coverage.missingMonths.length > 0) {
    log(`[PE-Hist] Missing critical months: ${coverage.missingMonths.join(', ')}
months ago`);
    log(`[PE-Hist] Recommendation: ${coverage.recommendation}`);
  }

  const prices = priceHistory.sort((a,b)=> new Date(b.date).getTime()-new
Date(a.date).getTime());
  const qAsc = quarters.slice().sort((a,b)=> new Date(a.periodEnd).getTime()-new
Date(b.periodEnd).getTime());

  // Build monthly EPS TTM series
  const monthlyEpsTtm = {};
  for (let i=0;i<qAsc.length;i++){
    const w = qAsc.slice(Math.max(0,i-3), i+1);
    if (w.length===4){
      const ni = w.map(q=>q.net_income).filter(v=>v!=null);
```

```
      const sh = w.map(q=>q.shares_diluted).filter(v=>v!=null);
      if (ni.length===4 && sh.length===4){
        const eps = ni.reduce((s,v)=>s+v,0) / (sh.reduce((s,v)=>s+v,0)/4);
        if (eps!=null && eps>0){
          const d=new Date(w[3].periodEnd); const
key=d.toISOString().slice(0,7);
          monthlyEpsTtm[key]=eps;
          log(`[PE-Hist] EPS TTM for ${key}: $${eps.toFixed(4)}`);
        }
      }
    }
  }

  const MIN_EPS = 0.01;

  const peFrom = (price, epsTTM) => {
    if (!price || !epsTTM || epsTTM < MIN_EPS) return null;
    return price / epsTTM;
  };

  // Robust month-end selection function
  function peAt(monthsAgo) {
    const targetDate = new Date();
    targetDate.setMonth(targetDate.getMonth() - monthsAgo);

    const monthKey = `${targetDate.getFullYear()}-${String(targetDate.getMonth()
+ 1).padStart(2, '0')}`;

    // Find the closest EPS TTM for this period
    const keys = Object.keys(monthlyEpsTtm).sort().reverse();
    const epsKey = keys.find(k => k <= monthKey);

    if (!epsKey) {
      log(`[PE-Hist] No EPS TTM found for ${monthsAgo}m ago (${monthKey})`);
      return null;
    }

    const eps = monthlyEpsTtm[epsKey];

    // Find month-end price using robust selection
    const monthEndPrice = pickMonthEnd(prices, targetDate);

    if (!monthEndPrice) {
      log(`[PE-Hist] No month-end price found for ${monthsAgo}m ago ($
{monthKey})`);
      return null;
    }

    // Use split-adjusted close (not dividend-adjusted adj_close) for P/E
calculations
    const pe = peFrom(monthEndPrice.close, eps);

    log(`[PE-Hist] ${monthsAgo}m ago: price=$${monthEndPrice.close?.toFixed(2)}
(${monthEndPrice.date}), eps=$${eps.toFixed(4)}, pe=${pe?.toFixed(2)}x`);

    return pe;
  }

  const pe_12m = peAt(12);
  const pe_24m = peAt(24);
  const pe_36m = peAt(36);

  log(`[PE-Hist] Historical P/E summary: PE_12m=${pe_12m?.toFixed(2)}, PE_24m=$
{pe_24m?.toFixed(2)}, PE_36m=${pe_36m?.toFixed(2)}`);
```

```
  // Validate results
  const validCount = [pe_12m, pe_24m, pe_36m].filter(pe => pe !== null).length;
  log(`[PE-Hist] ${validCount}/3 historical P/E values successfully
calculated`);

  return {
    pe_ttm_12m_ago: pe_12m,
    pe_ttm_24m_ago: pe_24m,
    pe_ttm_36m_ago: pe_36m,
  };
}

// ---------- DB upsert ----------
async function upsertFinnhubMetrics(base44, ticker, metrics, log) {
  const existing = await
base44.asServiceRole.entities.Metrics.filter({ ticker_symbol: ticker });

  // Always-overwrite list for fast-moving / UI-critical fields
  const ALWAYS_UPDATE = new Set([
    'eps_ttm','pe_ttm','pe_12m','pe_24m','pe_36m','current_pe',
    'cfo_ttm','capex_ttm','sbc_ttm','depreciation_ttm','ebit_ttm','ebitda_ttm',
    'net_income_ttm','cash','total_debt','equity','total_assets'
  ]);

  const clean = Object.fromEntries(Object.entries(metrics).filter(([,v]) => v !
== null && v !== undefined));

  if (existing.length > 0) {
    const row = existing[0];
    const updates = {};
    for (const [k,v] of Object.entries(clean)) {
      if (ALWAYS_UPDATE.has(k) || row[k] === null || row[k] === undefined)
updates[k] = v;
    }
    if (Object.keys(updates).length) {
      const src = row.data_source || '';
      updates.data_source = src.includes('finnhub') ? src : (src ? `${src}
+finnhub` : 'finnhub');
      await base44.asServiceRole.entities.Metrics.update(row.id, updates);
      log(`[DB] Updated ${ticker} with fields: ${Object.keys(updates).join(',
')}`);
    } else {
      log(`[DB] No fields changed for ${ticker}.`);
    }
  } else {
    await base44.asServiceRole.entities.Metrics.create({
      ticker_symbol: ticker,
      as_of_date: new Date().toISOString().slice(0,10),
      data_source: 'finnhub',
      ...clean
    });
    log(`[DB] Created new metrics record for ${ticker}.`);
  }
}

// ---------- Latest price helper (from your PricesHistory table) ----------
async function getLatestPrice(base44, ticker, log) {
  const latest = await
base44.asServiceRole.entities.PricesHistory.filter({ ticker, limit: 1, order:
'date.desc' });
  if (latest?.[0]) {
    log(`[Price] Latest price for ${ticker}: ${latest[0].close_adj} ($
{latest[0].date})`);
```

```
    return latest[0].close_adj;
  }
  return null;
}

// ---------- Server ----------
Deno.serve(async (req) => {
  const base44 = createClientFromRequest(req);
  const logs = [];
  const log = (msg) => logs.push(`[${new Date().toISOString().split('T')
[1].slice(0,-1)}] ${msg}`);

  if (!(await base44.auth.isAuthenticated())) {
    return new Response(JSON.stringify({ error:'Unauthorized' }),
{ status:401 });
  }

  try {
    const { ticker } = await req.json();
    if (!ticker) throw new Error('Ticker is required');

    const T = ticker.trim().toUpperCase();
    log(`[START] Running Finnhub ETL for ${T}`);

    const [qData, aData] = await Promise.all([
      fetchQuarterlyFinancials(T, log),
      fetchAnnualFinancials(T, log)
    ]);

    const quarters = normalizeAndQuarterizeData(qData, aData, log);
    if (quarters.length < 4) {
      log(`[Finnhub][Warn] Not enough Finnhub quarterly data for ${T} (found $
{quarters.length}). Will process what is available.`);
    }

    const ttm = buildTTMMetrics(quarters, log);
    const { net_income_ttm, cfo_ttm, eps_ttm } = ttm;

    if (!Number.isFinite(net_income_ttm) || !Number.isFinite(cfo_ttm)) {
      log('[Finnhub][Warn] Invalid basic TTM metrics (NI, CFO). Dependent
calculations will be skipped.');
    }
    if (!Number.isFinite(eps_ttm) || eps_ttm <= 0) {
      log('[Finnhub][Warn] Invalid EPS TTM. P/E calculations will be skipped.');
    }

    const historicalPE = await calculateHistoricalPE_DB(base44, T, quarters,
log);

    // Compute latest price & current PE
    const priceToday = await getLatestPrice(base44, T, log);
    const pe_ttm = (priceToday && eps_ttm > 0) ? (priceToday / eps_ttm) : null;

    // Map to Projection Tool field names
    const projectionFields = {
      eps_ttm,                       // core for UI
      pe_ttm,                        // current PE (TTM)
      pe_12m: historicalPE.pe_ttm_12m_ago,
      pe_24m: historicalPE.pe_ttm_24m_ago,
      pe_36m: historicalPE.pe_ttm_36m_ago,
      current_pe: pe_ttm,
    };

    const allMetrics = { ...ttm, ...projectionFields };
```

```
    const fcf_ttm = (ttm.cfo_ttm != null && ttm.capex_ttm != null) ?
(ttm.cfo_ttm + ttm.capex_ttm) : null;

    const cov = {
      CFO: ttm.cfo_ttm!=null ? 'Y':'N',
      CapEx: ttm.capex_ttm!=null ? 'Y':'N',
      SBC: ttm.sbc_ttm!=null ? 'Y':'N',
      Dep: ttm.depreciation_ttm!=null ? 'Y':'N',
      Cash: ttm.cash!=null ? 'Y':'N',
      ShortDebt: quarters[0]?.short_debt!=null ? 'Y':'N',
      LongDebt: quarters[0]?.long_debt!=null ? 'Y':'N',
      TotalDebt: ttm.total_debt!=null ? 'Y':'N',
      Equity: ttm.equity!=null ? 'Y':'N',
      TotalAssets: ttm.total_assets!=null ? 'Y':'N',
      EBIT: ttm.ebit_ttm!=null ? 'Y':'N',
      NI: ttm.net_income_ttm!=null ? 'Y':'N',
      EPS_TTM: ttm.eps_ttm!=null ? 'Y':'N',
      PE_TTM: pe_ttm!=null ? 'Y':'N',
      PE_12M: projectionFields.pe_12m!=null ? 'Y':'N',
      PE_24M: projectionFields.pe_24m!=null ? 'Y':'N',
      PE_36M: projectionFields.pe_36m!=null ? 'Y':'N',
    };
    log(`[Coverage-Finnhub] ${Object.entries(cov).map(([k,v])=>`${k}=$
{v}`).join(', ')}`);

    await upsertFinnhubMetrics(base44, T, allMetrics, log);

    log(`[Verify] cfo_ttm=${ttm.cfo_ttm}, capex_ttm=${ttm.capex_ttm}, fcf_ttm=$
{fcf_ttm}, cash=${ttm.cash}, total_debt=${ttm.total_debt}, equity=${ttm.equity},
ebit_ttm=${ttm.ebit_ttm}, depreciation_ttm=${ttm.depreciation_ttm}, ebitda_ttm=$
{ttm.ebitda_ttm}, net_income_ttm=${ttm.net_income_ttm}, eps_ttm=$
{eps_ttm?.toFixed(4)}, pe_ttm=${pe_ttm?.toFixed(2)}, pe_12m=$
{projectionFields.pe_12m?.toFixed(2)}, pe_24m=$
{projectionFields.pe_24m?.toFixed(2)}, pe_36m=$
{projectionFields.pe_36m?.toFixed(2)}`);

    const quartersProcessed = quarters.length;
    const metricsPopulated =
['CFO','CapEx','SBC','Dep','Cash','ShortDebt','LongDebt','TotalDebt','Equity','T
otalAssets','EBIT','NI','EPS_TTM','PE_TTM','PE_12M','PE_24M','PE_36M']
      .map(k => cov[k]).filter(v => v==='Y').length;

    // A) Standardized success response
    return new Response(JSON.stringify({
        ok: true,
        success: true,
        message: `Finnhub ETL processed ${quartersProcessed} quarters for $
{T}.`, // Use T for ticker as it's the uppercase one used in logs
        quarters_processed: quartersProcessed,
        metrics_populated: metricsPopulated,
        summary: {
            quarters_processed: quartersProcessed,
            metrics_populated: metricsPopulated,
            data_source: 'finnhub',
        },
        logs
    }), {
        status: 200,
        headers: { 'Content-Type': 'application/json' }
    });
  } catch (error) {
    log(`[FinnhubETL][FATAL] ${error.message}`);
    console.error(error);
```

```
    // A) Standardized error response
    return new Response(JSON.stringify({
        ok: false,
        success: false,
        error: `Finnhub ETL failed: ${error.message}`,
        message: `Could not process Finnhub fundamentals: ${error.message}`,
        logs
    }), {
        status: 500,
        headers: { 'Content-Type': 'application/json' }
    });
  }
});



import { createClientFromRequest } from 'npm:@base44/sdk@0.5.0';

Deno.serve(async (req) => {
    const base44 = createClientFromRequest(req);

    if (!(await base44.auth.isAuthenticated())) {
        return new Response(JSON.stringify({ error: 'Unauthorized' }), {
            status: 401,
            headers: { 'Content-Type': 'application/json' }
        });
    }

    try {
        const { ticker } = await req.json();
        if (!ticker) {
            return new Response(JSON.stringify({ error: 'Ticker required' }), {
                status: 400,
                headers: { 'Content-Type': 'application/json' }
            });
        }

        // Use the secret FINNHUB_API_KEY
        const FINNHUB_KEY = Deno.env.get("FINNHUB_API_KEY");
        if (!FINNHUB_KEY) {
            throw new Error("FINNHUB_API_KEY environment variable is not set");
        }

        const url = `https://finnhub.io/api/v1/stock/financials-reported?
symbol=${ticker}&token=${FINNHUB_KEY}`;

        console.log(`Fetching financials for ${ticker}`);

        const response = await fetch(url);
        const responseText = await response.text();

        if (!response.ok) {
            console.error(`Finnhub API error: ${response.status} $
{response.statusText}`, responseText);
            throw new Error(`Finnhub API error: ${response.status} $
{response.statusText}`);
        }

        let data;
        try {
            data = JSON.parse(responseText);
        } catch (e) {
```

```
            console.error(`Finnhub returned non-JSON response for ${ticker}:`,
responseText);
            return new Response(JSON.stringify({
                error: 'Invalid JSON response from Finnhub',
                responseText: responseText
            }), {
                status: 502, // Bad Gateway
                headers: { 'Content-Type': 'application/json' }
            });
        }

        console.log(`Finnhub financials response for ${ticker}: $
{data.data?.length || 0} filings`);

        return new Response(JSON.stringify({
            ticker,
            finjson: data
        }), {
            status: 200,
            headers: { 'Content-Type': 'application/json' }
        });

    } catch (error) {
        console.error('Error fetching financials:', error);
        return new Response(JSON.stringify({ error: error.message }), {
            status: 500,
            headers: { 'Content-Type': 'application/json' }
        });
    }
});



import { createClientFromRequest } from 'npm:@base44/sdk@0.5.0';

function corsHeaders() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "POST, GET, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type, Authorization",
    "Access-Control-Allow-Credentials": "true",
    "Content-Type": "application/json",
  };
}

async function getCandlesData(input = {}) {
  try {
    const ticker = String(input?.ticker ?? input?.symbol ??
'').trim().toUpperCase();
    if (!/^[A-Z.\-]{1,10}$/.test(ticker)) {
      return { status: 'invalid_symbol', message: 'Bad ticker format' };
    }

    const token = Deno.env.get("FINNHUB_API_KEY") ?? "";
    const USE_SANDBOX = (Deno.env.get("FINNHUB_USE_SANDBOX") === "true") ||
token.startsWith("sandbox_");
    const BASE = USE_SANDBOX ? "https://sandbox.finnhub.io" :
"https://finnhub.io";

    const now = Math.floor(Date.now() / 1000);
    const windows = [5 * 365 * 24 * 60 * 60, 365 * 24 * 60 * 60, 90 * 24 * 60 *
60];
    let lastPlanError = null;
```

```
    for (const windowSeconds of windows) {
      const from = now - windowSeconds;
      const url = new URL(`${BASE}/api/v1/stock/candle`);
      url.searchParams.set("symbol", ticker);
      url.searchParams.set("resolution", "D");
      url.searchParams.set("from", String(from));
      url.searchParams.set("to", String(now));
      url.searchParams.set("token", token);

      try {
        const resp = await fetch(url.toString(), { method: "GET" });
        const http_status = resp.status;
        const text = await resp.text();
        let data = {};
        try {
          data = text ? JSON.parse(text) : {};
        } catch (e) {
          // Hard fail on parse error, as it's unexpected
          return { status: "http_error", message: "Failed to parse Finnhub
response", debug: { windowSeconds, http_status, bodySnippet: text.slice(0,
120) }};
        }

        if (resp.ok && data.s === 'ok' && Array.isArray(data.t) &&
Array.isArray(data.c)) {
            const daily_prices = data.t.map((ts, i) => ({
              date: new Date(ts * 1000).toISOString().slice(0, 10),
              close_adj: Number(data.c[i]),
            })).filter(r => Number.isFinite(r.close_adj));

            if (daily_prices.length > 0) {
              return { status: "ok", http_status, finnhub_status: "ok", ticker,
count: daily_prices.length, daily_prices, windowSeconds };
            }
        }

        if (data.s === 'no_data') {
          continue; // Try next smaller window
        }

        if (http_status === 403 || http_status === 429) {
          lastPlanError = { windowSeconds, http_status, finnhub_status:
data.s ?? 'unknown' };
          continue; // Try next smaller window, might be plan-restricted
        }

        // For other non-2xx errors, fail fast
        if (!resp.ok) {
           return { status: "http_error", http_status, message: "HTTP error from
Finnhub", debug: { bodySnippet: text.slice(0,120) }};
        }
      } catch (fetchErr) {
        // Network-level error
        return { status: "exception", message: String(fetchErr.message ||
fetchErr), debug: { windowSeconds } };
      }
    }

    // If loop finishes, all windows failed
    if (lastPlanError) {
      return { status: "plan_restricted", http_status:
lastPlanError.http_status, message: "Candles blocked by plan/rate limit", debug:
{ triedWindows: windows.map(w => `${w/86400}d`) } };
```

```
    }

    return { status: "no_data", message: "Finnhub returned no data for all tried
windows", debug: { triedWindows: windows.map(w => `${w/86400}d`) } };

  } catch (err) {
    return { status: "exception", message: String(err.message || err) };
  }
}

Deno.serve(async (req) => {
  // Handle preflight to avoid 405
  if (req.method === 'OPTIONS') {
    return new Response(null, { status: 204, headers: corsHeaders() });
  }

  if (req.method !== 'POST' && req.method !== 'GET') {
    return new Response(JSON.stringify({ error: 'Method Not Allowed' }),
{ status: 405, headers: corsHeaders() });
  }

  // Auth (for GET we still require auth cookie/session)
  const base44 = createClientFromRequest(req);
  if (!(await base44.auth.isAuthenticated())) {
    return new Response(JSON.stringify({ error: 'Unauthorized' }), { status:
401, headers: corsHeaders() });
  }

  let input = {};
  if (req.method === 'POST') {
    try { input = await req.json(); } catch {
      return new Response(JSON.stringify({ error: 'Bad JSON body' }), { status:
400, headers: corsHeaders() });
    }
  } else if (req.method === 'GET') {
    const u = new URL(req.url);
    input = { ticker: (u.searchParams.get('ticker') || '').toUpperCase() };
  }

  const result = await getCandlesData(input);
  return new Response(JSON.stringify(result), { status: 200, headers:
corsHeaders() });
});



import { createClientFromRequest } from 'npm:@base44/sdk@0.5.0';

function corsHeaders() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "POST, GET, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type, Authorization",
    "Access-Control-Allow-Credentials": "true",
    "Content-Type": "application/json",
  };
}

Deno.serve(async (req) => {
  if (req.method === 'OPTIONS') {
    return new Response(null, { status: 204, headers: corsHeaders() });
  }
```

```
  // No auth needed for a simple health check

  try {
    const url = "https://query1.finance.yahoo.com/v8/finance/chart/AAPL?
range=1mo&interval=1d&includeAdjustedClose=true";
    const resp = await fetch(url);
    const http_status = resp.status;
    const data = await resp.json();

    if (!resp.ok) {
      return new Response(JSON.stringify({ ok: false, http_status, message:
"Yahoo API returned non-200 status", debug: data }), { status: 200, headers:
corsHeaders() });
    }

    const result = data?.chart?.result?.[0];
    const lastIndex = result?.timestamp?.length - 1;
    if (!result || lastIndex < 0) {
       return new Response(JSON.stringify({ ok: false, http_status, message:
"Unexpected response structure", debug: data }), { status: 200, headers:
corsHeaders() });
    }

    const lastAdjClose = result.indicators.adjclose[0].adjclose[lastIndex];
    const lastTs = result.timestamp[lastIndex];

    return new Response(JSON.stringify({ ok: true, http_status, sample: { c:
lastAdjClose, t: lastTs } }), { status: 200, headers: corsHeaders() });

  } catch (err) {
    return new Response(JSON.stringify({ ok: false, error: String(err.message ||
err), debug: { details: "Function execution exception"} }), { status: 500,
headers: corsHeaders() });
  }
});



import { createClientFromRequest } from 'npm:@base44/sdk@0.5.0';

function corsHeaders() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "POST, GET, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type, Authorization",
    "Access-Control-Allow-Credentials": "true",
    "Content-Type": "application/json",
  };
}

Deno.serve(async (req) => {
  // Handle preflight
  if (req.method === 'OPTIONS') {
    return new Response(null, { status: 204, headers: corsHeaders() });
  }

  // Auth
  const base44 = createClientFromRequest(req);
  if (!(await base44.auth.isAuthenticated())) {
    return new Response(JSON.stringify({ error: 'Unauthorized' }), { status:
401, headers: corsHeaders() });
  }
```

```
  try {
    const FINNHUB_KEY = Deno.env.get("FINNHUB_API_KEY") ||
"d2g4fd1r01qkv5nfnjvgd2g4fd1r01qkv5nfnk00";
    const url = `https://finnhub.io/api/v1/quote?symbol=AAPL&token=$
{FINNHUB_KEY}`;

    const resp = await fetch(url);
    const http_status = resp.status;
    const body = await resp.json();

    if (!resp.ok) {
      return new Response(JSON.stringify({ ok: false, http_status, body }),
{ status: 200, headers: corsHeaders() });
    }

    return new Response(JSON.stringify({ ok: true, http_status, body }),
{ status: 200, headers: corsHeaders() });

  } catch (err) {
    return new Response(JSON.stringify({ ok: false, error: String(err.message ||
err) }), { status: 500, headers: corsHeaders() });
  }
});
```

```
import { createClientFromRequest } from 'npm:@base44/sdk@0.5.0';

function corsHeaders() {
  return {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Methods": "POST, GET, OPTIONS",
    "Access-Control-Allow-Headers": "Content-Type, Authorization",
    "Access-Control-Allow-Credentials": "true",
    "Content-Type": "application/json",
  };
}

function nz(value) {
    // Safely get raw number, return null if not a finite number
    return Number.isFinite(value?.raw) ? value.raw : null;
}

Deno.serve(async (req) => {
  if (req.method === 'OPTIONS') {
    return new Response(null, { status: 204, headers: corsHeaders() });
  }

  let ticker = "";
  try {
    if (req.method === 'POST') {
      const body = await req.json();
      ticker = body.ticker;
    } else {
      ticker = new URL(req.url).searchParams.get('ticker');
    }
  } catch {
    return new Response(JSON.stringify({ ok: false, status: 400, reason:
'Invalid request body' }), { status: 400, headers: corsHeaders() });
  }
```

```
  if (!ticker) {
    return new Response(JSON.stringify({ ok: false, status: 400, reason: 'Ticker
required' }), { status: 400, headers: corsHeaders() });
  }

  // Added console.log as per outline
  console.log(`[Yahoo] Fetching fundamentals for ${ticker}`);

  const modules =
"price,summaryDetail,defaultKeyStatistics,financialData,calendarEvents,assetProf
ile,earnings";
  const url = `https://query2.finance.yahoo.com/v10/finance/quoteSummary/$
{ticker}?modules=${modules}`;

  const maxRetries = 2; // Initial try + 1 retry
  const backoff = 400; // ms
  const timeout = 8000; // 8 seconds

  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), timeout);

    try {
      const resp = await fetch(url, { signal: controller.signal });
      clearTimeout(timeoutId);

      if (!resp.ok) {
        if (resp.status >= 500 && attempt < maxRetries) {
          await new Promise(r => setTimeout(r, backoff));
          continue; // Retry on 5xx errors
        }
        return new Response(JSON.stringify({ ok: false, status: resp.status,
reason: `HTTP error: ${resp.statusText}`, ticker }), { status: 200, headers:
corsHeaders() });
      }

      const data = await resp.json();
      const summary = data?.quoteSummary?.result?.[0];

      if (!summary) {
        return new Response(JSON.stringify({ ok: false, status: 404, reason: "No
summary data in response", ticker }), { status: 200, headers: corsHeaders() });
      }

      const fields = {
          pe_ttm: nz(summary.summaryDetail?.trailingPE) ??
nz(summary.defaultKeyStatistics?.trailingPE),
          pe_fwd: nz(summary.summaryDetail?.forwardPE) ??
nz(summary.financialData?.forwardPE),
          peg_5y: nz(summary.defaultKeyStatistics?.pegRatio),
          ev_ebitda: nz(summary.defaultKeyStatistics?.enterpriseToEbitda),
          div_yield_pct: (nz(summary.summaryDetail?.dividendYield) ?? 0) * 100,
          payout_ratio: nz(summary.summaryDetail?.payoutRatio),
          beta: nz(summary.summaryDetail?.beta),
          market_cap: nz(summary.price?.marketCap),
          shares_out: nz(summary.defaultKeyStatistics?.sharesOutstanding),
          debt: nz(summary.financialData?.totalDebt),
          cash: nz(summary.financialData?.totalCash),
          profile: { // Include profile data
              name: summary.price?.longName || summary.price?.shortName ||
ticker,
              exchange: summary.price?.exchangeName,
              sector: summary.assetProfile?.sector,
          }
```

```
      };

      // Blacklist ratio fields that should only come from
computeFundamentalMetrics
      const RATIO_BLACKLIST = new Set([
        'roic_pct', 'fcf_margin_pct', 'cfo_to_ni', 'fcf_to_ebit',
'accruals_ratio',
        'margin_stdev_5y_pct', 'buyback_yield_pct', 'debt_to_equity',
        'netdebt_to_ebitda', 'interest_coverage_x',
        'pe_12m', 'pe_24m', 'pe_36m', 'pe_5y_low', 'pe_5y_high', 'pe_5y_median'
      ]);

      // Clean the data and apply blacklist
      const cleanData = {};
      for (const [key, value] of Object.entries(fields)) { // Iterating over
'fields' as per current structure
        if (RATIO_BLACKLIST.has(key)) {
          console.log(`[Yahoo] Skipping blacklisted ratio field: ${key}`);
          continue;
        }

        // Preserve objects (like 'profile') which are not intended to be
filtered by isNaN
        if (typeof value === 'object' && value !== null) {
          cleanData[key] = value;
        } else {
          // Apply the original outline's condition for scalar values
          if (value !== null && value !== undefined && !isNaN(value)) {
            cleanData[key] = value;
          }
        }
      }

      // Added console.log as per outline
      console.log(`[Yahoo] Successfully processed ${ticker}, saved $
{Object.keys(cleanData).length} fields`);

      // Modified response as per outline, ensuring 'fields' object is included
and CORS headers are kept
      return new Response(JSON.stringify({
        ok: true,
        status: 200, // Keep status 200 for successful responses
        ticker,
        fields: cleanData, // Changed to cleanData as the primary 'fields'
payload
        fields_saved: Object.keys(cleanData),
        fields_blacklisted: [...RATIO_BLACKLIST]
      }), {
        status: 200, // Keep status 200 for successful responses
        headers: corsHeaders() // Ensure CORS headers are included
      });

    } catch(e) {
      clearTimeout(timeoutId);
      if (e.name === 'AbortError') {
        return new Response(JSON.stringify({ ok: false, status: 408, reason:
'Request timed out', ticker }), { status: 200, headers: corsHeaders() });
      }
      if (attempt < maxRetries) {
          await new Promise(r => setTimeout(r, backoff));
          continue;
      }
      return new Response(JSON.stringify({ ok: false, status: 500, reason:
`Exception: ${e.message}`, ticker }), { status: 200, headers: corsHeaders() });
```

```
    }
  }
});




import { createClientFromRequest } from 'npm:@base44/sdk@0.5.0';

// --- START: SHARED HELPERS & SESSION MANAGEMENT ---
const isFiniteNum = (x) => typeof x === 'number' && Number.isFinite(x);
const asArray = (v) => Array.isArray(v) ? v : [];
const keyCount = (obj) => (obj && typeof obj === 'object') ?
Object.keys(obj).length : 0;

// --- START: ROBUST YAHOO SESSION & CRUMB MANAGEMENT ---
const yahooSession = {
  cookie: null,
  crumb: null,
  crumbTs: 0,
  userAgent: 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/124.0.0.0 Safari/537.36',
};
async function ensureCrumb(ticker = 'AAPL', log) {
  if (yahooSession.crumb && (Date.now() - yahooSession.crumbTs < 10 * 60 *
1000)) { return yahooSession.crumb; }
  const headers = { 'User-Agent': yahooSession.userAgent, 'Accept':
'text/html,application/json;q=0.9,*/*;q=0.8', 'DNT': '1' };
  try { const primeResponse = await fetch(`https://finance.yahoo.com/quote/$
{encodeURIComponent(ticker)}`, { headers }); const cookieHeader =
primeResponse.headers.get('set-cookie'); if (cookieHeader) yahooSession.cookie =
cookieHeader; } catch (e) { log(`[crumb] cookie priming failed: $
{e.message}`); }
  const crumbHeaders = { ...headers, ...(yahooSession.cookie && { 'Cookie':
yahooSession.cookie }) };
  for (const host of ['query2', 'query1']) {
     try {
       const crumbResponse = await fetch(`https://$
{host}.finance.yahoo.com/v1/test/getcrumb`, { headers: crumbHeaders });
      const text = await crumbResponse.text();
      const body = (text || '').trim();
      if (crumbResponse.status === 200 && body) {
         yahooSession.crumb = body;
         yahooSession.crumbTs = Date.now();
         return body;
       }
     } catch (e) { log(`[crumb] S1: ${host} error: ${e.message}`); }
   }
   throw { stage: 'session', message: 'Could not obtain Yahoo crumb.' };
}
async function fetchSourceQ(ticker, log) {
    log("[Source Q] Fetching quoteSummary data...");
    const crumb = await ensureCrumb(ticker, log);
    const modules = ["incomeStatementHistoryQuarterly",
"cashflowStatementHistoryQuarterly", "balanceSheetHistoryQuarterly",
"incomeStatementHistory", "cashflowStatementHistory", "balanceSheetHistory",
"financialData", "defaultKeyStatistics", "price", "summaryDetail"].join(",");
    const url = new
URL(`https://query2.finance.yahoo.com/v10/finance/quoteSummary/${ticker}`);
    url.search = new URLSearchParams({ modules, crumb, formatted: "false", lang:
"en-US", region: "US", corsDomain: "finance.yahoo.com" }).toString();
    const headers = { 'Cookie': yahooSession.cookie || '', 'User-Agent':
yahooSession.userAgent };
```

```
    const response = await fetch(url.toString(), { headers });
    if (!response.ok) throw new Error(`quoteSummary returned $
{response.status}`);
    const json = await response.json();
    if (!json?.quoteSummary?.result?.[0]) {
      log('[Source Q] Empty/invalid quoteSummary payload; continuing with
minimal fields.');
      return {};
    }
    log(`[Source Q] Successfully fetched quoteSummary data`);
    return json.quoteSummary.result[0];
}
async function fetchSourceF(ticker, log) { return null;}
async function fetchSourceT(ticker, log) { return null;}

// --- C) DB UPSERT WITH PRESERVED STRINGS ---
async function upsertWithGuard(base44, entityName, matchFields, payload, log) {
    const existing = await
base44.asServiceRole.entities[entityName].filter(matchFields);
    const clean = {};
    const skippedKeys = [];
    const alwaysAllow = new Set(['as_of_date', 'data_source', 'ticker_symbol']);

    for (const [k, v] of Object.entries(payload)) {
        if (alwaysAllow.has(k) && typeof v === 'string') {
            clean[k] = v;
            continue;
        }
        if (isFiniteNum(v)) {
            clean[k] = v;
        } else {
            skippedKeys.push(k);
        }
    }

    if (existing.length > 0) {
        const id = existing[0].id;
        delete clean.ticker_symbol;
        if (Object.keys(clean).length > 0) {
            await base44.asServiceRole.entities[entityName].update(id, clean);
            log(`[DB Upsert] Updated: [${Object.keys(clean).join(', ')}]`);
        } else {
            log(`[DB Upsert] No valid fields to update.`);
        }
        if (skippedKeys.length > 0) log(`[DB Upsert] Skipped null/invalid
fields: [${skippedKeys.join(', ')}]`);
        return "updated";
    } else {
        const createPayload = { ...matchFields, ...clean };
        await base44.asServiceRole.entities[entityName].create(createPayload);
        log(`[DB Upsert] Created: [${Object.keys(clean).join(', ')}]`);
        if (skippedKeys.length > 0) log(`[DB Upsert] Skipped null/invalid fields
on create: [${skippedKeys.join(', ')}]`);
        return "inserted";
    }
}


// --- NUMERIC EXTRACTION WITH .RAW SUPPORT ---
function num(n) {
    if (typeof n?.raw === 'number') return n.raw;
    if (typeof n?.raw === 'string' && n.raw.trim() !== '' && !
isNaN(Number(n.raw))) return Number(n.raw);
    if (typeof n === 'number') return n;
```

```
    if (typeof n === 'string' && n.trim() !== '' && !isNaN(Number(n))) return
Number(n);
    return null;
}
const pickNum = (...candidates) => {
  for (const c of candidates) {
    const v = num(c);
    if (typeof v === 'number' && Number.isFinite(v)) return v;
  }
  return null;
};
function toISODate(d) {
    if (!d) return null;
    let timestamp;
    if (typeof d?.raw === 'number') timestamp = d.raw;
    else if (typeof d === 'number') timestamp = d;
    else if (typeof d === 'string') return d.slice(0, 10); // Assume ISO string
already
    else return null;
    const dt = new Date(timestamp * 1000);
    return isFinite(dt.getTime()) ? dt.toISOString().slice(0, 10) : null;
}

function normalizeSourceQ(sourceQ, log) {
    if (!sourceQ || typeof sourceQ !== 'object') return {};
    const incQ =
asArray(sourceQ?.incomeStatementHistoryQuarterly?.incomeStatementHistory);
    const cfsQ =
asArray(sourceQ?.cashflowStatementHistoryQuarterly?.cashflowStatements);
    const bssQ =
asArray(sourceQ?.balanceSheetHistoryQuarterly?.balanceSheetStatements);
    const quarters = {};
    [...incQ, ...cfsQ, ...bssQ].forEach((stmt) => {
         const date = toISODate(stmt?.endDate);
         if (date) {
             if (!quarters[date]) quarters[date] = {};
             Object.assign(quarters[date], stmt);
         }
     });
    log(`[Source Q] Normalized ${keyCount(quarters)} quarters`);
    return quarters;
}
function normalizeSourceF(sourceF, log) {
    if (!sourceF || !sourceF.timeseries || !
asArray(sourceF.timeseries.result).length) return {};
    const quarters = {};
    asArray(sourceF.timeseries.result).forEach(series => {
        if (asArray(series.quarterly).length) {
            asArray(series.quarterly).forEach(entry => {
                 const date = toISODate(entry?.asOfDate || entry?.endDate);
                 if (date) {
                     if (!quarters[date]) quarters[date] = {};
                    const metaType = asArray(series?.meta?.type);
                    const metricName = metaType[0] || 'unknown';
                     quarters[date][metricName] = entry;
                 }
             });
        }
    });
    log(`[Source F] Normalized ${keyCount(quarters)} quarters`);
    return quarters;
}
function normalizeSourceT(sourceT, log) {
```

```
    if (!sourceT || !sourceT.timeseries || !
asArray(sourceT.timeseries.result).length) return {};
    const quarters = {};
    asArray(sourceT.timeseries.result).forEach(series => {
        const metaType = asArray(series?.meta?.type);
        const metricName = metaType[0] || 'unknown';
        if (asArray(series.quarterly).length) {
            asArray(series.quarterly).forEach(entry => {
                 const date = toISODate(entry?.asOfDate || entry?.endDate);
                 if (date) {
                     if (!quarters[date]) quarters[date] = {};
                     quarters[date][metricName] = entry;
                 }
             });
        }
    });
    log(`[Source T] Normalized ${keyCount(quarters)} quarters`);
    return quarters;
}
function mergeMultiSource(quartersQ, quartersF, quartersT, log) {
    const allDates = [...new Set([
        ...Object.keys(quartersQ || {}),
        ...Object.keys(quartersF || {}),
        ...Object.keys(quartersT || {})
    ])].sort().reverse();

    const recentDates = allDates.slice(0, 4);

    const merged = recentDates.map(date => {
        const qData = quartersQ[date] || {};
        const fData = quartersF[date] || {};
        const tData = quartersT[date] || {};

        const result = {
            as_of_date: date,
            // Income Statement
            revenue: num(qData.totalRevenue) || num(fData.Revenue) ||
num(tData.revenue),
            net_income: num(qData.netIncome) || num(fData.NetIncome) ||
num(tData.net_income),
            ebit: num(qData.ebit) || num(fData.OperatingIncome) ||
num(tData.ebit),
            depreciation: num(qData.depreciation) ||
num(fData.DepreciationAndAmortization) ||
num(tData.depreciation_and_amortization),
            sbc: num(qData.stockBasedCompensation) ||
num(fData.StockBasedCompensation) || num(tData.stock_based_compensation),
            interest_expense: num(qData.interestExpense) ||
num(fData.InterestExpense) || num(tData.interest_expense),

            // Cash Flow Statement
            cfo: num(qData.totalCashFromOperatingActivities) ||
num(fData.OperatingCashFlow) || num(tData.cash_flow_from_operating_activities),
            capex: num(qData.capitalExpenditures) ||
num(fData.CapitalExpenditures) || num(tData.capital_expenditure),

            // Balance Sheet
            cash: num(qData.cash) || num(fData.CashAndCashEquivalents) ||
num(tData.cash_and_cash_equivaents),
            short_debt: num(qData.shortTermDebt) || num(fData.ShortTermDebt) ||
num(tData.short_term_debt),
            long_debt: num(qData.longTermDebt) || num(fData.LongTermDebt) ||
num(tData.long_term_debt),
```

```
            equity: num(qData.totalStockholderEquity) || num(fData.TotalEquity)
|| num(tData.total_equity),
            total_assets: num(qData.totalAssets) || num(fData.TotalAssets) ||
num(tData.total_assets),
        };
        const coverage = {};
        for (const key in result) {
            if (key === 'as_of_date') continue;
            let source = 'None';
            if (result[key] === num(qData[key])) source = 'Q';
            else if (result[key] === num(fData[key])) source = 'F';
            else if (result[key] === num(tData[key])) source = 'T';
            coverage[key] = source;
        }
        result.coverage = coverage;
        return result;
    }).filter(q => q.as_of_date);

    if (asArray(merged).length > 0) {
         const latestCoverage = merged[0].coverage || {};
        const entries = Object.entries(latestCoverage);
        const coverageReport = entries.map(([field, source]) => `${field}:$
{source}`).join(', ');
        const fieldsPopulated = entries.filter(([,src]) => src !==
'None').length;
        const totalFields = entries.length;
         const coveragePercent = Math.round((fieldsPopulated / (totalFields ||
1)) * 100);
         log(`[Coverage] ${coverageReport}`);
        log(`[Coverage] fields_populated = ${fieldsPopulated}/${totalFields ||
1} (${coveragePercent || 0}%)`);
    }
    return merged;
}

// --- C) ROBUST TTM BUILDER ---
function buildTTMFromMerged(merged, sourceQ, ticker, log) {
    if (asArray(merged).length < 4) {
         log(`[TTM] Warning: Found only ${asArray(merged).length} quarters of
data. TTM values may be incomplete.`);
    }

    const sum4 = (vals) => {
        const nums = (vals || []).filter(v => typeof v === 'number' &&
Number.isFinite(v));
        return nums.length === 4 ? nums.reduce((a, b) => a + b, 0) : null;
    };

    const pick = (k) => merged.map(q => (typeof q[k] === 'number' ? q[k] :
null));

    const revenue_ttm = sum4(pick('revenue'));
    const cfo_ttm = sum4(pick('cfo'));
    const capex_ttm = sum4(pick('capex'));
    const sbc_ttm = sum4(pick('sbc'));
    const ebit_ttm = sum4(pick('ebit'));
    const depreciation_ttm = sum4(pick('depreciation'));
    const ebitda_ttm = (isFiniteNum(ebit_ttm) &&
isFiniteNum(depreciation_ttm)) ? ebit_ttm + depreciation_ttm : null;
    const fcf_ttm = (isFiniteNum(cfo_ttm) && isFiniteNum(capex_ttm)) ? (cfo_ttm
- Math.abs(capex_ttm)) : null;

    const latest = merged[0] || {};
```

```
    const payload = {
        ticker_symbol: ticker,
        as_of_date: new Date().toISOString().slice(0, 10),
        data_source: 'yahoo:multi_source_v1',

        price_current: pickNum(sourceQ?.price?.regularMarketPrice,
sourceQ?.summaryDetail?.regularMarketPreviousClose),
        revenue_ttm,
        cfo_ttm,
        capex_ttm,
        sbc_ttm,
        fcf_ttm,
        ebit_ttm,
        depreciation_ttm,
        ebitda_ttm,
        net_income_ttm: sum4(pick('net_income')),
        interest_expense_ttm: sum4(pick('interest_expense')),

        cash: latest.cash,
        total_debt: (latest.short_debt !== null || latest.long_debt !== null) ?
(latest.short_debt || 0) + (latest.long_debt || 0) : null,
        equity: latest.equity,
        total_assets: latest.total_assets,

        market_cap: num(sourceQ?.price?.marketCap),
        shares_out: num(sourceQ?.defaultKeyStatistics?.sharesOutstanding),
        pe_fwd: num(sourceQ?.summaryDetail?.forwardPE),
        pe_ttm: num(sourceQ?.summaryDetail?.trailingPE),
        beta_5y: num(sourceQ?.defaultKeyStatistics?.beta),

        enterprise_value: pickNum(
          sourceQ?.defaultKeyStatistics?.enterpriseValue,
          sourceQ?.financialData?.enterpriseValue
        ),
        enterprise_to_ebitda: pickNum(
          sourceQ?.defaultKeyStatistics?.enterpriseToEbitda,
          sourceQ?.financialData?.enterpriseToEbitda
        ),

        current_pe: pickNum(
          sourceQ?.summaryDetail?.trailingPE,
          sourceQ?.defaultKeyStatistics?.trailingPE
        )
    };

    if (payload.enterprise_value != null && payload.ebitda_ttm != null &&
payload.ebitda_ttm !== 0) {
        const ratio = payload.enterprise_value / payload.ebitda_ttm;
        if (Number.isFinite(ratio)) payload.ev_ebitda = ratio;
    } else if (payload.enterprise_to_ebitda != null) {
        payload.ev_ebitda = payload.enterprise_to_ebitda;
    }

    return payload;
}

// --- MAIN DENO SERVER ---
Deno.serve(async (req) => {
    const base44 = createClientFromRequest(req);
    const logs = [];
    const log = (msg) => logs.push(`[${new Date().toISOString().split('T')
[1].slice(0, -1)}] ${msg}`);
    if (!(await base44.auth.isAuthenticated())) {
```

```
        return new Response(JSON.stringify({ ok: false, success: false, error:
'Unauthorized' }), { status: 401 });
    }

    try {
         const { ticker, force_backfill = false } = await req.json();
         if (!ticker) throw { stage: 'input', message: 'Ticker is required' };
         const tickerUpper = ticker.trim().toUpperCase();
         log(`[START] Yahoo Fundamentals ETL for ${tickerUpper}
(force_backfill=${force_backfill})`);

        const sourceQ = await fetchSourceQ(tickerUpper, log);

        // --- SAVE QUARTERLY FINANCIALS HISTORY ---
        const quarterlyFinancials = [];
        const annualFinancials = [];

        // Process quarterly data
        const allQuarterDates = [...new Set([

...asArray(sourceQ?.incomeStatementHistoryQuarterly?.incomeStatementHistory).map
(s => toISODate(s.endDate)),

...asArray(sourceQ?.cashflowStatementHistoryQuarterly?.cashflowStatements).map(s
=> toISODate(s.endDate)),

...asArray(sourceQ?.balanceSheetHistoryQuarterly?.balanceSheetStatements).map(s
=> toISODate(s.endDate))
        ])].filter(Boolean).sort().reverse(); // Most recent first

        log(`[Quarterly] Found ${allQuarterDates.length} unique quarterly dates
to process.`);

        for (const date of allQuarterDates) {
            const inc =
asArray(sourceQ?.incomeStatementHistoryQuarterly?.incomeStatementHistory).find(s
=> toISODate(s.endDate) === date) || {};
            const cfs =
asArray(sourceQ?.cashflowStatementHistoryQuarterly?.cashflowStatements).find(s
=> toISODate(s.endDate) === date) || {};
            const bss =
asArray(sourceQ?.balanceSheetHistoryQuarterly?.balanceSheetStatements).find(s =>
toISODate(s.endDate) === date) || {};

            // Robust interest expense with fallback and sign normalization
            let interestExp = num(inc.interestExpense) ?? num(cfs.interestPaid);
            if (interestExp != null) interestExp = Math.abs(interestExp);

            const quarterlyPayload = {
                ticker: tickerUpper,
                period_end: date,
                freq: 'quarterly',
                source: 'yahoo',
                as_of_date: new Date().toISOString().slice(0,10),

                // Income Statement
                revenue: num(inc.totalRevenue),
                net_income: num(inc.netIncome),
                ebit: num(inc.ebit),
                interest_expense: interestExp,
                depreciation: num(cfs.depreciation),
                stock_based_compensation: num(inc.stockBasedCompensation) ||
num(cfs.stockBasedCompensation),
                shares_diluted: num(inc.dilutedAverageShares),
```

```
                eps_diluted: num(inc.dilutedEps),

                // Cash Flow Statement
                cfo: num(cfs.totalCashFromOperatingActivities),
                capex: num(cfs.capitalExpenditures),
                fcf: isFiniteNum(num(cfs.totalCashFromOperatingActivities)) &&
isFiniteNum(num(cfs.capitalExpenditures)) ?
                     num(cfs.totalCashFromOperatingActivities) -
Math.abs(num(cfs.capitalExpenditures)) : null,

                // Balance Sheet
                cash: num(bss.cash),
                short_debt: num(bss.shortLongTermDebt),
                long_debt: num(bss.longTermDebt),
                total_debt: (isFiniteNum(num(bss.shortLongTermDebt)) ||
isFiniteNum(num(bss.longTermDebt))) ?
                            (num(bss.shortLongTermDebt) || 0) +
(num(bss.longTermDebt) || 0) : null,
                stockholder_equity: num(bss.totalStockholderEquity),
                total_assets: num(bss.totalAssets),
            };
            quarterlyFinancials.push(quarterlyPayload);
        }

        // Process annual data (if available)
        const allAnnualDates = [...new Set([

...asArray(sourceQ?.incomeStatementHistory?.incomeStatementHistory).map(s =>
toISODate(s.endDate)),

...asArray(sourceQ?.cashflowStatementHistory?.cashflowStatements).map(s =>
toISODate(s.endDate)),

...asArray(sourceQ?.balanceSheetHistory?.balanceSheetStatements).map(s =>
toISODate(s.endDate))
        ])].filter(Boolean).sort().reverse();

        log(`[Annual] Found ${allAnnualDates.length} unique annual dates to
process.`);

        for (const date of allAnnualDates) {
            const inc =
asArray(sourceQ?.incomeStatementHistory?.incomeStatementHistory).find(s =>
toISODate(s.endDate) === date) || {};
            const cfs =
asArray(sourceQ?.cashflowStatementHistory?.cashflowStatements).find(s =>
toISODate(s.endDate) === date) || {};
            const bss =
asArray(sourceQ?.balanceSheetHistory?.balanceSheetStatements).find(s =>
toISODate(s.endDate) === date) || {};

            let interestExp = num(inc.interestExpense) ?? num(cfs.interestPaid);
            if (interestExp != null) interestExp = Math.abs(interestExp);

            const annualPayload = {
                ticker: tickerUpper,
                period_end: date,
                freq: 'annual',
                source: 'yahoo',
                as_of_date: new Date().toISOString().slice(0,10),

                revenue: num(inc.totalRevenue),
                net_income: num(inc.netIncome),
                ebit: num(inc.ebit),
```

```
                interest_expense: interestExp,
                depreciation: num(cfs.depreciation),
                stock_based_compensation: num(inc.stockBasedCompensation) ||
num(cfs.stockBasedCompensation),
                shares_diluted: num(inc.dilutedAverageShares),
                eps_diluted: num(inc.dilutedEps),

                cfo: num(cfs.totalCashFromOperatingActivities),
                capex: num(cfs.capitalExpenditures),
                fcf: isFiniteNum(num(cfs.totalCashFromOperatingActivities)) &&
isFiniteNum(num(cfs.capitalExpenditures)) ?
                     num(cfs.totalCashFromOperatingActivities) -
Math.abs(num(cfs.capitalExpenditures)) : null,

                cash: num(bss.cash),
                short_debt: num(bss.shortLongTermDebt),
                long_debt: num(bss.longTermDebt),
                total_debt: (isFiniteNum(num(bss.shortLongTermDebt)) ||
isFiniteNum(num(bss.longTermDebt))) ?
                            (num(bss.shortLongTermDebt) || 0) +
(num(bss.longTermDebt) || 0) : null,
                stockholder_equity: num(bss.totalStockholderEquity),
                total_assets: num(bss.totalAssets),
            };
            annualFinancials.push(annualPayload);
        }

        // Upsert quarterly financials
        let qUpsertedCount = 0;
        for (const record of quarterlyFinancials) {
            const { ticker, period_end, freq, ...data } = record;
            const matchFields = { ticker, period_end, freq };
            const cleanData =
Object.fromEntries(Object.entries(data).filter(([,v]) => v != null));

            if (Object.keys(cleanData).length === 0) continue;

            const existing = await
base44.asServiceRole.entities.FinancialsHistory.filter(matchFields);
            if(existing.length > 0) {
                await
base44.asServiceRole.entities.FinancialsHistory.update(existing[0].id,
cleanData);
            } else {
                await
base44.asServiceRole.entities.FinancialsHistory.create({ ...matchFields,
...cleanData });
            }
            qUpsertedCount++;
        }

        // Upsert annual financials
        let aUpsertedCount = 0;
        for (const record of annualFinancials) {
            const { ticker, period_end, freq, ...data } = record;
            const matchFields = { ticker, period_end, freq };
            const cleanData =
Object.fromEntries(Object.entries(data).filter(([,v]) => v != null));

            if (Object.keys(cleanData).length === 0) continue;

            const existing = await
base44.asServiceRole.entities.FinancialsHistory.filter(matchFields);
            if(existing.length > 0) {
```

```
                await
base44.asServiceRole.entities.FinancialsHistory.update(existing[0].id,
cleanData);
            } else {
                await
base44.asServiceRole.entities.FinancialsHistory.create({ ...matchFields,
...cleanData });
            }
            aUpsertedCount++;
        }

        log(`[FinancialsHistory] Completed upsert for ${qUpsertedCount}
quarterly + ${aUpsertedCount} annual reports.`);

        // Save current metrics to Metrics table
        const metricsPayload = buildTTMFromMerged([], sourceQ || {},
tickerUpper, log);
        await upsertWithGuard(base44, 'Metrics', { ticker_symbol: tickerUpper },
metricsPayload, log);

        return new Response(JSON.stringify({
            ok: true,
            success: true,
            message: `Yahoo fundamentals processed for ${tickerUpper}.`,
            quarterly_count: qUpsertedCount,
            annual_count: aUpsertedCount,
            summary: { ticker: tickerUpper, fields:
Object.keys(metricsPayload).filter(k => metricsPayload[k] != null) },
            logs
        }), { status: 200, headers: { 'Content-Type': 'application/json' } });

    } catch (error) {
        const errorMessage = (error && error.message) ? error.message :
String(error || 'Unknown error');
        log(`[FATAL] ${error?.stage || 'unknown'}: ${errorMessage}`);
        return new Response(JSON.stringify({ ok: false, success: false, stage:
error?.stage || 'unknown', error: errorMessage, message: `Failed during Yahoo
ETL: ${errorMessage}`, logs }), { status: 500, headers: { 'Content-Type':
'application/json' } });
    }
});




import { createClientFromRequest } from 'npm:@base44/sdk@0.5.0';

// --- START: SHARED HELPERS ---

// --- Rate Limiting & Pacing ---
const yahooTokenBucket = { tokens: 2, lastRefill: Date.now(), refillRate: 1 /
600, maxTokens: 2 };
const dbTokenBucket = { tokens: 3, lastRefill: Date.now(), refillRate: 3 / 1000,
maxTokens: 3 };

function refillBucket(bucket) {
  const now = Date.now();
  const elapsed = now - bucket.lastRefill;
  const tokensToAdd = Math.floor(elapsed * bucket.refillRate);

  if (tokensToAdd > 0) {
    bucket.tokens = Math.min(bucket.maxTokens, bucket.tokens + tokensToAdd);
    bucket.lastRefill = now;
```

```
  }
}

async function waitForToken(bucket, log, logPrefix) {
  while (bucket.tokens < 1) {
    refillBucket(bucket);
    if (bucket.tokens < 1) {
      log(`${logPrefix}[RateLimit] waiting for token, current=$
{bucket.tokens.toFixed(2)}`);
      await new Promise(resolve => setTimeout(resolve, 100));
    }
  }
  bucket.tokens -= 1;
  refillBucket(bucket);
}

// --- Exponential Backoff ---
async function withBackoff(fn, options) {
    const { retries, base, cap, jitter, retryOn, onRetry } = options;
    for (let attempt = 1; attempt <= retries; attempt++) {
        try {
            return await fn();
        } catch (error) {
            const status = error.response?.status || error.status || 0;
            const code = error.code || 'unknown';
            const isRetryable = retryOn.includes(status) || retryOn.some(c =>
code.includes(c));

            if (!isRetryable || attempt === retries) {
                throw error; // Throw original error if not retryable or final
attempt
            }

            const delay = Math.min(cap, base * Math.pow(1.5, attempt - 1));
            const jitteredDelay = jitter ? Math.round(delay * (1 +
(Math.random() - 0.5) * 0.4)) : delay;

            onRetry(attempt, jitteredDelay, { status, code });
            await new Promise(resolve => setTimeout(resolve, jitteredDelay));
        }
    }
}

// --- Session & Fetching ---
let yahooSession = { cookie: null, crumb: null, acquiredAt: 0, inFlight: null };
const SESSION_TTL_MS = 60 * 60 * 1000;
function sessionIsFresh() { return yahooSession.cookie && yahooSession.crumb &&
(Date.now() - yahooSession.acquiredAt) < SESSION_TTL_MS; }
async function parseJsonOrHtml(response, log) { const text = await
response.text(); if (text.startsWith('<')) { log(`[YahooFetch] Non-JSON (HTML)
status=${response.status}`); return { ok: false, isHtml: true, status:
response.status, text }; } try { const json = JSON.parse(text);
log(`[YahooFetch] json OK`); return { ok: true, isJson: true, json, status:
response.status }; } catch (e) { log(`[YahooFetch] Non-JSON detected (Parse
Error) status=${response.status}`); return { ok: false, parseError: true,
status: response.status, text }; } }
async function fetchCrumbWithBackoff(log) {
    const maxAttempts = 6; const baseDelayMs = 1000;
    for (let attempt = 1; attempt <= maxAttempts; attempt++) {
        try {
            await waitForToken(yahooTokenBucket, log, '[Backfill]');
            const cookieResponse = await fetch('https://fc.yahoo.com/',
{ headers: { 'User-Agent': 'Mozilla/5.0' } });
```

```
            if (cookieResponse.status === 429 || cookieResponse.status >= 500)
throw new Error(`Cookie fetch failed: status=${cookieResponse.status}`);
            const cookieHeader = cookieResponse.headers.get('set-cookie'); if (!
cookieHeader) throw new Error("No session cookie");
            await waitForToken(yahooTokenBucket, log, '[Backfill]');
            const crumbResponse = await
fetch('https://query1.finance.yahoo.com/v1/test/getcrumb', { headers:
{ 'cookie': cookieHeader, 'User-Agent': 'Mozilla/5.0' } });
            if (crumbResponse.status === 429 || crumbResponse.status >= 500)
throw new Error(`Crumb fetch failed: status=${crumbResponse.status}`);
            const crumb = await crumbResponse.text(); if (!crumb || crumb.length
< 5) throw new Error("Invalid crumb");
            yahooSession.cookie = cookieHeader; yahooSession.crumb = crumb;
yahooSession.acquiredAt = Date.now();
            log("[Backfill][Stage:session] cookie OK"); log("[Backfill]
[Stage:session] crumb OK");
            return;
        } catch (error) {
            if (attempt === maxAttempts) throw { stage: 'session', code: 429,
message: 'Crumb fetch failed after retries', cause: error.message };
            const delay = Math.min(6000, baseDelayMs * Math.pow(1.5, attempt -
1));
            log(`[Backfill][Retry ${attempt}/${maxAttempts}] stage=session
status=${error.message.match(/status=(\d+)/)?.[1] || 'n/a'} sleep=${delay}ms`);
            await new Promise(resolve => setTimeout(resolve, delay));
        }
    }
}
async function ensureSession(log) { log("[Backfill][Stage:session] getSession
start"); if (sessionIsFresh()) { log("[Backfill][Stage:session] cache hit");
return; } log("[Backfill][Stage:session] cache miss"); if
(yahooSession.inFlight) { log("[Backfill][Stage:session] waiting for in-flight
session"); await yahooSession.inFlight; return; } yahooSession.inFlight =
fetchCrumbWithBackoff(log); try { await yahooSession.inFlight; } finally
{ yahooSession.inFlight = null; } }
async function signedFetch(path, params, log) {
    await ensureSession(log);
    const url = new URL(`https://query1.finance.yahoo.com${path}`);
    const finalParams = { ...params, crumb: yahooSession.crumb };
    Object.entries(finalParams).forEach(([key, value]) =>
url.searchParams.append(key, value));
    await waitForToken(yahooTokenBucket, log, '[Backfill]');
    const response = await fetch(url.toString(), { headers: { 'cookie':
yahooSession.cookie, 'User-Agent': 'Mozilla/5.0' } });
    const bodyText = await response.text();
    log(`[Backfill][Stage:fetch] <- ${path} status=${response.status} len=$
{bodyText.length}`);
    if (!response.ok) {
      const error = new Error(`Yahoo fetch failed with status $
{response.status}`);
      error.status = response.status;
      throw error;
    }
    const mockResponse = { status: response.status, text: () =>
Promise.resolve(bodyText) };
    return parseJsonOrHtml(mockResponse, log);
}
function buildChartRequest(opts) { const { symbol, interval = '1d', events =
'div%2Csplits', range, period1, period2 } = opts; if (!symbol) throw new
Error('Symbol required'); const urlPath = `/v8/finance/chart/${symbol}`; const
params = { interval, events, includeAdjustedClose: 'true' }; if (period1 &&
period2) { params.period1 = period1.toString(); params.period2 =
period2.toString(); } else if (range) { params.range = range; } else { throw new
Error('Time window required'); } return { urlPath, params }; }
```

```
// --- ETL LOGIC ---
const toYMD = (d) => d ? new Date(d).toISOString().slice(0, 10) : null;
const numOrNull = (v) => (v === null || v === undefined || Number.isNaN(+v)) ?
null : +v;

async function fetchYahooData(ticker, log) {
    log("[Backfill] Using YahooSession v1");
    const { urlPath, params } = buildChartRequest({ symbol: ticker, range:
'5y' });
    const chartResult = await signedFetch(urlPath, params, log);
    if (!chartResult.ok || !chartResult.json?.chart?.result?.[0]) {
        throw { stage: 'fetch', code: chartResult.status || 500, message: `Could
not fetch price history for ${ticker}`, cause: chartResult.text?.slice(0, 120)
|| 'No chart data' };
    }
    return { priceData: chartResult.json.chart.result[0] };
}

function normalizePrices(ticker, priceData, log) {
    const timestamps = Array.isArray(priceData?.timestamp) ? priceData.timestamp
: [];
    if (!timestamps.length) return [];
    log(`[Backfill][Stage:parse] processing ${timestamps.length} price
timestamps`);

    const quotes = priceData.indicators.quote[0];
    const adjCloses = priceData.indicators.adjclose[0].adjclose;

    const prices = timestamps.map((ts, i) => ({
        ticker,
        date: toYMD(new Date(ts * 1000)),
        open: numOrNull(quotes?.open?.[i]),
        high: numOrNull(quotes?.high?.[i]),
        low: numOrNull(quotes?.low?.[i]),
        close: numOrNull(quotes?.close?.[i]),       // Split-adjusted only
(Yahoo "Close")
        close_adj: numOrNull(adjCloses?.[i]),       // Dividend & split-adjusted
(Yahoo "Adj Close")
        volume: numOrNull(quotes?.volume?.[i]),
        source: 'yahoo',
        as_of_date: toYMD(new Date())
    })).filter(p => p.date && p.close_adj !== null && p.close !== null); //
Require both prices

    log(`[Backfill][Stage:parse] normalized ${prices.length} valid price
points`);
    return prices;
}

async function upsertPrices(base44, prices, log) {
    const safePrices = Array.isArray(prices) ? prices : [];
    if (safePrices.length === 0) {
        log("[Backfill][Stage:db] No price data to upsert.");
        return { inserted: 0, updated: 0, skipped: 0 };
    }

    // AUTH DIAGNOSTICS for service role access
    log("[Backfill][Stage:db] Initializing service role database
operations...");

    const serviceClient = base44.asServiceRole; // Use the service role client
for elevated permissions
```

```
    try {
        // Test service role connectivity
        const testQuery = await serviceClient.entities.PricesHistory.filter({},
null, 1);
        log(`[Backfill][Stage:db] Service role connectivity: ${testQuery ?
'CONNECTED' : 'FAILED'}`);

    } catch (serviceTestError) {
        log(`[Backfill][Stage:db] Service role test failed: $
{serviceTestError.message}`);
        throw {
            stage: 'db',
            code: 'SERVICE_ROLE_FAILED',
            message: 'Service role database access failed',
            cause: serviceTestError.message
        };
    }

    const ticker = safePrices[0].ticker;
    const BATCH_SIZE = 25; // Reduced for better error handling
    const batches = [];
    for (let i = 0; i < safePrices.length; i += BATCH_SIZE) {
        batches.push(safePrices.slice(i, i + BATCH_SIZE));
    }
    log(`[Backfill][Stage:db] Processing ${batches.length} batches of max $
{BATCH_SIZE} rows each`);

    let totalInserted = 0, totalUpdated = 0, totalSkipped = 0;

    for (let i = 0; i < batches.length; i++) {
        const batch = batches[i];
        const dates = batch.map(p => p.date);

        try {
            const existing = await serviceClient.entities.PricesHistory.filter({
                ticker,
                date: { '$in': dates }
            });

            const safeExisting = Array.isArray(existing) ? existing : [];
            const existingMap = new Map(safeExisting.map(p => [p.date, p]));
            const toInsert = batch.filter(row => !existingMap.has(row.date));

            if (toInsert.length > 0) {
                try {
                    // Wait for DB token before write
                    await waitForToken(dbTokenBucket, log, '[Backfill][Stage:db]
batch ' + (i+1) + '/' + batches.length);

                    // Perform the write with backoff using service role
                    await withBackoff(
                        () =>
serviceClient.entities.PricesHistory.bulkCreate(toInsert),
                        {
                            retries: 6,
                            base: 300,
                            cap: 6000,
                            jitter: true,
                            retryOn: [429, 503, 'FetchError', 'ETIMEDOUT',
'ECONNRESET'],
                            onRetry: (n, wait, err) => log(`[Backfill][Retry $
{n}] stage=db status=${err.status||'n/a'} sleep=${wait}ms`)
                        }
                    );
```

```
                    totalInserted += toInsert.length;
                    log(`[Backfill][Stage:db] Batch ${i+1}/${batches.length}
complete: inserted ${toInsert.length}`);

                } catch (dbError) {
                    const status = dbError.response?.status || dbError.status ||
500;
                    log(`[Backfill FATAL] stage=db status=${status} message=DB
write failed on batch ${i+1} after retries.`);

                    // Enhanced unauthorized error handling
                    if (status === 401 || status === 403) {
                        log(`[Backfill FATAL] UNAUTHORIZED ERROR - Service role
access denied`);
                        log(`[Backfill FATAL] Check RLS policies on
PricesHistory table for service_role`);

                        throw {
                            stage: 'db',
                            code: 'UNAUTHORIZED',
                            message: `Unauthorized service role access on batch
${i+1}. Check RLS policies.`,
                            cause: dbError.message,
                            resumeHint: `Run failed at batch ${i+1}/$
{batches.length}. ${totalInserted} rows inserted.`
                        };
                    }

                    // Throw structured error to be caught by the main handler
                    throw {
                        stage: 'db',
                        code: status,
                        message: `DB write for batch ${i+1} failed after
retries.`,
                        cause: dbError.message,
                        resumeHint: `Run failed at batch ${i+1}/$
{batches.length}. ${totalInserted} rows inserted.`
                    };
                }
            }

            const skipped = batch.length - toInsert.length;
            totalSkipped += skipped;

        } catch (batchError) {
            // Re-throw structured errors
            if (batchError.stage) {
                throw batchError;
            }

            log(`[Backfill][Stage:db] Batch ${i+1} error: $
{batchError.message}`);
            totalSkipped += batch.length;
        }
    }

    log(`[Backfill][Stage:db] All batches complete. Total new rows inserted: $
{totalInserted}.`);
    return { inserted: totalInserted, updated: totalUpdated, skipped:
totalSkipped };
}

// In-process metrics computation helper (no HTTP calls)
async function computeDerivedMetricsInProcess(base44, ticker, log) {
```

```
    log(`[Backfill][Stage:finalize] computing price-derived metrics for $
{ticker}`);

    try {
        // Get price history from DB
        const fiveYearsAgo = new Date(new Date().setFullYear(new
Date().getFullYear() - 5)).toISOString().slice(0, 10);

        const tickerPrices = await base44.entities.PricesHistory.filter({
            ticker: ticker,
            date: { $gte: fiveYearsAgo }
        }, '-date', 2000);

        if (!Array.isArray(tickerPrices) || tickerPrices.length < 252) {
            log(`[Backfill][Stage:finalize] insufficient price data for metrics
(${tickerPrices?.length || 0} records)`);
            return { metricsComputed: 0 };
        }

        const derivedMetrics = {};

        // Calculate max drawdown
        let peak = tickerPrices[0].close_adj;
        let maxDrawdown = 0;
        for (const price of tickerPrices) {
            if (price.close_adj > peak) peak = price.close_adj;
            const drawdown = (peak - price.close_adj) / peak;
            if (drawdown > maxDrawdown) maxDrawdown = drawdown;
        }
        derivedMetrics.maxdrawdown_5y_pct = maxDrawdown * 100;

        // Update metrics table
        const existingMetrics = await
base44.entities.Metrics.filter({ ticker_symbol: ticker });
        const updatePayload = {
            ...derivedMetrics,
            data_source: 'derived:backfill',
            as_of_date: toYMD(new Date())
        };

        if (existingMetrics.length > 0) {
            await base44.entities.Metrics.update(existingMetrics[0].id,
updatePayload);
        } else {
            await base44.entities.Metrics.create({ ticker_symbol: ticker,
...updatePayload });
        }

        log(`[Backfill][Stage:finalize] updated $
{Object.keys(derivedMetrics).length} derived metrics for ${ticker}`);
        return { metricsComputed: Object.keys(derivedMetrics).length };

    } catch (error) {
        // Don't fail the entire backfill for metrics computation failure
        log(`[Backfill][Stage:finalize] metrics computation failed: $
{error.message}`);
        return { metricsComputed: 0, metricsError: error.message };
    }
}

// --- MAIN DENO SERVER ---
let activeRuns = new Map();
Deno.serve(async (req) => {
    const base44 = createClientFromRequest(req);
```

```
    if (!(await base44.auth.isAuthenticated())) return new
Response(JSON.stringify({ success: false, message: 'Unauthorized' }), { status:
401 });

    const logs = [];
    const log = (msg) => logs.push(`[${new Date().toISOString().split('T')
[1].slice(0, -1)}] ${msg}`);

    let tickerUpper = '';
    let currentStage = 'init';

    try {
        const { ticker } = await req.json();
        tickerUpper = ticker.trim().toUpperCase();

        // Stage: init
        currentStage = 'init';
        log(`[Backfill][Stage:init] starting full ETL for ${tickerUpper}...`);

        // Concurrency guard
        if (activeRuns.has(tickerUpper)) {
            log(`[Backfill][Guard] run locked for ${tickerUpper}`);
            await activeRuns.get(tickerUpper);
            return new Response(JSON.stringify({ success: true, message:
'Concurrent run completed', logs }), { status: 200 });
        }

        log(`[Backfill][Guard] run locked for ${tickerUpper}`);

        const runPromise = (async () => {
            let yahooResult, normalizedPrices, priceResult, metricsResult;

            try {
                // Stage: fetch
                currentStage = 'fetch';
                yahooResult = await fetchYahooData(tickerUpper, log);

                // Stage: parse
                currentStage = 'parse';
                normalizedPrices = normalizePrices(tickerUpper,
yahooResult.priceData, log);

                // Stage: db
                currentStage = 'db';
                priceResult = await upsertPrices(base44, normalizedPrices, log);

                // Stage: finalize (in-process metrics computation)
                currentStage = 'finalize';
                metricsResult = await computeDerivedMetricsInProcess(base44,
tickerUpper, log);

                // Stage: complete
                currentStage = 'complete';
                log(`[Backfill SUCCESS] Complete. Received: $
{normalizedPrices.length}, Inserted: ${priceResult.inserted}, Updated: $
{priceResult.updated}.`);

                return {
                    success: true,
                    message: `ETL Completed for ${tickerUpper}`,
                    logs,
                    summary: {
                        financials_stored: 0,
                        prices_stored: priceResult.inserted,
```

```
                        metrics_computed: metricsResult.metricsComputed || 0
                    }
                };

            } catch (stageError) {
                // Re-throw with current stage context if not already structured
                if (!stageError.stage) {
                    stageError.stage = currentStage;
                    stageError.code = stageError.status || stageError.code ||
500;
                }
                throw stageError;
            }
        })();

        activeRuns.set(tickerUpper, runPromise);
        const result = await runPromise;
        return new Response(JSON.stringify(result), { status: 200 });

    } catch (error) {
        const stage = error.stage || currentStage || 'unknown';
        const code = error.code || error.status || 500;
        const message = error.message || 'Unknown error occurred';

        log(`[Backfill FATAL] stage=${stage} code=${code} message=${message}`);

        return new Response(JSON.stringify({
            success: false,
            stage: stage,
            code: code,
            message: message,
            cause: error.cause || null,
            guidance: `ETL failed at ${stage}. ${error.resumeHint || ''}`,
            logs
        }), {
            status: code >= 400 && code < 600 ? code : 500,
            headers: { 'Content-Type': 'application/json' }
        });

    } finally {
        if (tickerUpper && activeRuns.has(tickerUpper)) {
            activeRuns.delete(tickerUpper);
            log(`[Backfill][Guard] run unlocked for ${tickerUpper}`);
        }
    }
});
```